Computers & Security 173 (2027) 105208

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Full length article

Evaluating security and inclusivity in LLM-generated code

Bilal Naqvi [a,*], Waleed Shahid [b], Janne Parkkila [a], Hammad Afzal [c]

[a] LUT University, Lappeenranta, Finland
[b] Royal Holloway University of London, United Kingdom
[c] University of Leicester, United Kingdom

ARTICLE INFO

Keywords:
Empirical studies
Human aspects
Inclusivity
Large Language Model
LLM
Security
Usability
Usable security

ABSTRACT

Despite developments in the human aspects of cybersecurity, inclusivity concerns, particularly related to neurodiversity, remain largely unexplored. This paper examines the role that Large Language Models (LLMs) can play in generating code that integrates inclusivity concerns in the development of security systems and services. To do so, a study was conducted using two user stories, each comprising three experimental cases that varied in the level of inclusivity specification from no specification in the first case to a detailed specification in the third During each case, the LLM was tasked to generate a code artifact that implemented the specified requirements. Furthermore, ten repeated code generations were performed in each case to minimize the impact of run-to-run stochastic variation. The generated code artifacts were then subjected to independent evaluations by six different LLMs using a rubric-based assessment. The highest-scoring artifacts based on the rubric-based LLM evaluations were subsequently evaluated by ten inclusivity and ten security experts, and the security assessment was further complemented by static and dynamic analysis using automated tools. The results indicate that greater specification detail is associated with improved inclusivity in LLM-generated code. Statistical comparisons using mixed-effects models revealed no statistically significant differences between human and LLM-based evaluations for inclusivity, whereas a significant difference was observed for security assessments. The findings have implications for research on integrating LLMs within secure software development contexts by highlighting the role of specification details and complementary strengths of human and LLM evaluators for assessing LLM-generated code.

1. Introduction

The human aspects of cybersecurity, interchangeably referred to as usable security, have been researched for nearly a quarter of a century (Naqvi and Smolander, 2024). Consequently, there have been developments in the domain of usable security with a primary focus on improving user experience while interacting with the security functions. However, these developments have mostly considered users as a monolithic group without regard to varying physical and cognitive abilities, literacies, and skills that the users possess (Renaud and Coles-Kemp, 2022).

Inclusivity is about recognizing and supporting users based on the differences in abilities, skills, and personal attributes, including those they are born with and those developed during their lifetime. In software engineering, the IEEE 25010:2023 standard on Systems and Software Engineering Systems and Software Quality Requirements and Evaluation (SQuaRE) defines inclusivity as the product's ability to be used by people from varying backgrounds. According to the standard, these backgrounds encompass different age groups, abilities, cultures, ethnicities, literacy levels, languages, economic and life situations (International Organization for Standardization (ISO) / International Electrotechnical Commission (IEC), 2023). Inclusivity in cybersecurity is an important dimension that impacts the right to access digital technologies for approximately 16% of the world's population (World Health Organization, 2023). More importantly, the key question is whether neurodivergent users can be blamed if their actions, or lack of action, result in a security breach. This question becomes more relevant and important to address as more security breaches attributed to human factors are reported every day (IBM, 2025).

Users of security systems and services could be having various forms of disabilities, including physical, sensory, cognitive, psychomotor, mental health, or a combination of these. However, the current work focuses on cognition-related disabilities in the context of users' cybersecurity interactions primarily because: (i) cognition is an enabler of the use of security functions and it directly impacts how users understand, memorize and interact with the information provided via

* Corresponding author.
E-mail address: syed.naqvi@lut.fi (B. Naqvi).

https://doi.org/10.1016/j.cose.2026.105208
Received 13 June 2026; Received in revised form 29 September 2026; Accepted 12 October 2026
Available online 29 October 2026
0167-4048/© 2026 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

the user interface, and (ii) cognitive disabilities are often overlooked in software systems (in general) compared to physical and sensory disabilities, primarily because they often go unnoticed, and their effects vary considerably among users, ranging from mild to profound levels (Renaud and Coles-Kemp, 2022; Naqvi et al., 2024). The main reasons for this oversight are a lack of understanding among security practitioners about neurodiversity concerns and a lack of policy directives on the topic (Renaud and Coles-Kemp, 2022). Moreover, the industry's perspective calls for an elaborate specification of the client's neurodiversity requirements along with multidisciplinary collaborations on the subject (Naqvi et al., 2024).

In cognitive science, neurodiversity is a term often used to refer to unique ways a human brain works (Liebel et al., 2024). Being neurodivergent means having brain functions different from the average "neurotypical" people (Naqvi et al., 2024). Thus, the term neurodivergent refers to people with cognitive challenges and limitations. Within neurodiversity, there are several unique conditions (also referred to as cognitive disabilities) such as intellectual disabilities, autism spectrum disorders (ASD), attention-deficit/hyperactivity disorder (ADHD), mental illness, brain injury, and stroke, among others. Cognitively demanding cybersecurity tasks are "usability hinders" for users, and these hindrances are often more severe for users with cognitive disabilities (Kävrestad et al., 2024). A task that is perceived as a nuisance by a neurotypical user may be impossible for a user with a cognitive disability to complete (Kävrestad et al., 2024).

Furthermore, the users' needs to enable a positive security interaction vary with the type of cognitive condition they have. For instance, users with ADHD often experience difficulties with sustained attention, task organization, and managing distractions, which in the context of cybersecurity can lead to challenges such as overlooking security warnings or failing to complete multi-factor authentication processes. In contrast, users with dyslexia may experience difficulties in reading comprehension and information processing speed, which can make understanding complex password policies or phishing awareness messages more challenging. Moreover, those with ASD typically face challenges in interpreting ambiguous language and adapting to unfamiliar interfaces, which can hinder their ability to follow abstract and unstructured security instructions (Daroff and Aminoff, 2014; Naqvi and Kavrestad, 2025).

Consideration of neurodiversity concerns in the design of secure software systems is not merely a quality goal, but is rather an equality concern. It is relevant to mention that security and usability are often considered as conflicting characteristics (Lennartsson et al., 2021; Naqvi et al., 2021); we extend this argument to state that inclusivity in cybersecurity requires extending usability further to enable neurodivergent users with various conditions to use the system. Therefore, maintaining a balance between security and inclusivity requirements is often a challenge for software developers (Naqvi et al., 2021). However, with large language models (LLMs) increasingly being used in secure software development contexts, it is important to examine how they balance these requirements, which can often be conflicting. This research explores the potential of LLMs in the development of security mechanisms that account for neurodiversity concerns. This is supported by the premise that LLMs have the potential to revolutionize industrial practices and automate some of the processes in the software development life-cycle (SDLC) (Shahid et al., 2025). These include generating inclusive requirements from requirement specification documents, or evaluating interfaces from the perspective of neurodivergent users' needs, while retaining code security (Shahid et al., 2025). In view of the goals of this research, the following research questions are considered:

1. RQ-1: How effectively can LLMs generate code that implements security and inclusivity requirements, particularly for neurodivergent users?
2. RQ-2: How does LLM-generated code perform when reviewed by humans and LLMs in terms of security robustness and inclusivity adequacy?

To investigate these questions, a study was conducted using two distinct user stories, each comprising three cases that progressed from minimal specification of inclusivity concerns to highly detailed representations. The LLM-generated code artifacts in each case, across both user stories, were independently reviewed by six LLMs, ten inclusivity experts, and ten security experts using a rubric-based assessment. Briefly, in line with RQ-1, the findings reveal that greater level of specification detail is associated with improved inclusivity in LLM-generated code. Moreover, in line with RQ-2, the statistical analyses using a mixed-effects model found no statistically significant difference between human and LLM evaluations of inclusivity, whereas a significant difference was observed in the security assessments. In addition to advancing inclusivity and promoting equitable access to technology, this research has broader implications for cybersecurity risk mitigation, as neurodivergent users may be more vulnerable to certain cyberattacks, and failing to address their needs can increase the attack surface.

The remainder of this paper is structured as follows. Section 2 presents the related work, Section 3 presents the methodology for the study, Section 4 presents the evaluation rubrics for inclusivity and security evaluations, Section 5 presents the findings, Section 6 presents the discussion, and Section 7 concludes the paper and outlines directions for future work.

2. Related work

Large Language Models (LLMs) have transformed many traditional and everyday approaches in various sectors. AI-based productivity tools are expected to contribute to a $1.5 trillion increase in global GDP by 2030 (Dohmke et al., 2023). Software development is one such key sector that has witnessed an abundant use of LLMs by developers, designers, and quality assurance resources alike. Moreover, both desktop and mobile applications containing LLM-generated code are already being used by customers. Much work on LLM-generated code has focused on security, whereas little explicit attention has been given to inclusivity and neurodiversity.

Many empirical studies have demonstrated that LLM-generated code might be functionally correct but contains security flaws and inconsistencies. Dora et al. (2025) identified critical security vulnerabilities in authentication, session management, and input validation related codes generated by popular LLMs, including ChatGPT, DeepSeek, Claude, Gemini, and Grok. The findings emphasized the need for human oversight and robust security assessment frameworks for the LLM-generated code to meet industry best practices. Black et al. (2024) also emphasize that, despite increasing development efficiency, LLMs do produce code with known security vulnerabilities. The research evaluated various prompt engineering strategies and demonstrated that appropriate prompt guidance significantly reduces vulnerabilities. Kharma et al. (2025) highlight that the security performance of LLM-generated code varies by language. They indicate the models often fail to adopt newer security features and still rely on outdated and insecure coding practices. Ramírez et al. (2024) identified seven types of security vulnerabilities in LLM-generated code after analyzing *3,104* studies from major digital libraries and finally selected *15* of these for detailed synthesis. Hajipour et al. (2024) developed a benchmark dataset of diverse insecure prompts to evaluate and compare the susceptibility of LLMs to generate code with known security vulnerabilities. Zhao et al. present SecureCoder Zhao et al. (2025), which integrates Retrieval-Augmented Generation with CWE-based classification to generate security-aware coding guidelines. They demonstrate that the proposed approach reduces vulnerabilities by over 65%. Andročec (2026) analyzed LLM-generated web applications using static, dynamic, and

manual techniques. The study found that explicit security requirements reduced the number and severity of security vulnerabilities. All these studies indicate that LLM-generated code commonly contains security weaknesses in areas like authentication, session management, input validation, cryptography, and the use of outdated APIs.

Human in the loop (HITL) has widely been presented as a way to curb the over-reliance on AI automation in code generation and keep developers in charge of the LLM-generated code. Pasuksmit et al. (2025) presented detailed research directions for improving evaluation frameworks for HITL software development tools. Sergeyuk et al. (2026) introduced the concept of Human-AI Experience on AI integration in Integrated Development Environments (IDEs). The research raised concerns about over-reliance and the quality of LLM-generated code, recommending strong user controls and audits. González et al. propose Human-in-the-Loop Decoding (HiLDE) (González et al., 2025), which allows users to view and influence key LLM decisions during code generation by reducing vulnerabilities and aligning with the user goals. Takerngsaksiri et al. (2025) proposed a Human-in-the-Loop LLM-based Software Development Agent Framework (HULA), which contains a human agent along with an AI Planner and Coding agents to improve the quality of LLM-generated code. Xue and Wang (2025) present a three-tier Human-in-the-Loop Intelligent Testing (HITL-IT) framework which combines human input with AI through continuous feedback to support efficient, reliable, and explainable testing, especially for safety-critical software.

It is worth highlighting that fewer research works have examined whether LLMs can produce accessible code. Aljedaani et al. (2024) evaluated ChatGPT-generated code based on WCAG 2.2 (Web Content Accessibility Guidelines) (Caldwell et al., 2008), and whether it can address accessibility issues in its own code and in open-source GitHub projects. They found that ChatGPT was able to fix about 70% of accessibility problems in the code it generated and about 73% of accessibility issues in third-party open-source project code. Suh et al. (2025) compare the accessibility of web code generated by GPT-4 and Qwen2.5-Coder with human-written code and found out that LLMs produce better accessibility for simple features like color contrast and alt text, but they struggle with more complex issues. Andruccioli et al. (2025) identified that LLMs can identify real accessibility violations in code, but they also produce some incorrect or unnecessary warnings, resulting in the need for a human review.

Existing research provides extensive comparative security evaluations of LLM-generated code, but does not yet offer empirical comparisons between the security and inclusivity of that code. Inclusivity is mostly discussed theoretically, without clear metrics or proper evaluation. There is very little research on how LLM-generated code supports neurodivergent user groups and their needs, highlighting a clear gap in research on how to measure and improve inclusivity in AI-assisted development. Table 1 summarizes the key research focus areas and their relation to this study.

3. Methodology

This study is an exploratory case study designed to examine how Large Language Models (LLMS) generate code that simultaneously addresses security and inclusivity requirements, particularly for neurodivergent users. The investigation was conducted in line with the research questions presented earlier and the following hypothesis.

1. Hypothesis (H1): LLMs can generate syntactically correct code that receives favorable security evaluation scores, but neurodiversity concerns would be underrepresented unless explicitly stated.
2. Hypothesis (H2a): There is a difference between human and LLM-based evaluators in their evaluation of inclusivity adequacy in the LLM-generated code.
3. Hypothesis (H2b): There is a difference between human and LLM-based evaluators in their evaluation of security robustness in LLM-generated code.

The experimental setup, as shown in Fig. 1, involved constructing three experimental cases for each user story, which varied in their level of inclusivity specification, from no specification in the first case to a detailed specification in the third. Furthermore, to account for the impact of run-to-run stochastic variation in LLM outputs, the coder LLM was tasked with generating a code artifact ten times in each case. The generated code artifacts were independently evaluated by six different LLMs, and subsequently by ten inclusivity experts and ten security experts. The user stories, along with key elements of the experimental setup, are discussed in the following subsections.

3.1. User stories and experimental design

The following two user stories were considered for this study featuring healthcare and online banking contexts, respectively.

User Story-1

Helena is a 67-year-old patient with chronic heart disease and a cognitive condition, ADHD. She calls the hospital to book an appointment to review her medication dosage. Due to updated security and privacy terms and conditions, she has to accept the updated conditions by logging in to her user account first for hospital authorities to be able to book an appointment for her. Authentication to her personal healthcare account requires a strong password, which she has forgotten; therefore, she needs to reset the password to be able to access the account and accept the updated security and privacy terms and conditions.

User Story-2

Marcus is a 54-year-old self-employed contractor who has just opened an account with an online bank. He manages his account through the bank's mobile web application, a responsive website he opens in his phone's browser. Following a regulatory update, the bank requires all customers to enrol in multi-factor authentication (MFA) before authorizing payments above a threshold. Marcus signs in, verifies his identity, sets up a time-based one-time passcode (OTP) authenticator, and securely stores a set of backup recovery codes. Marcus has dyslexia.

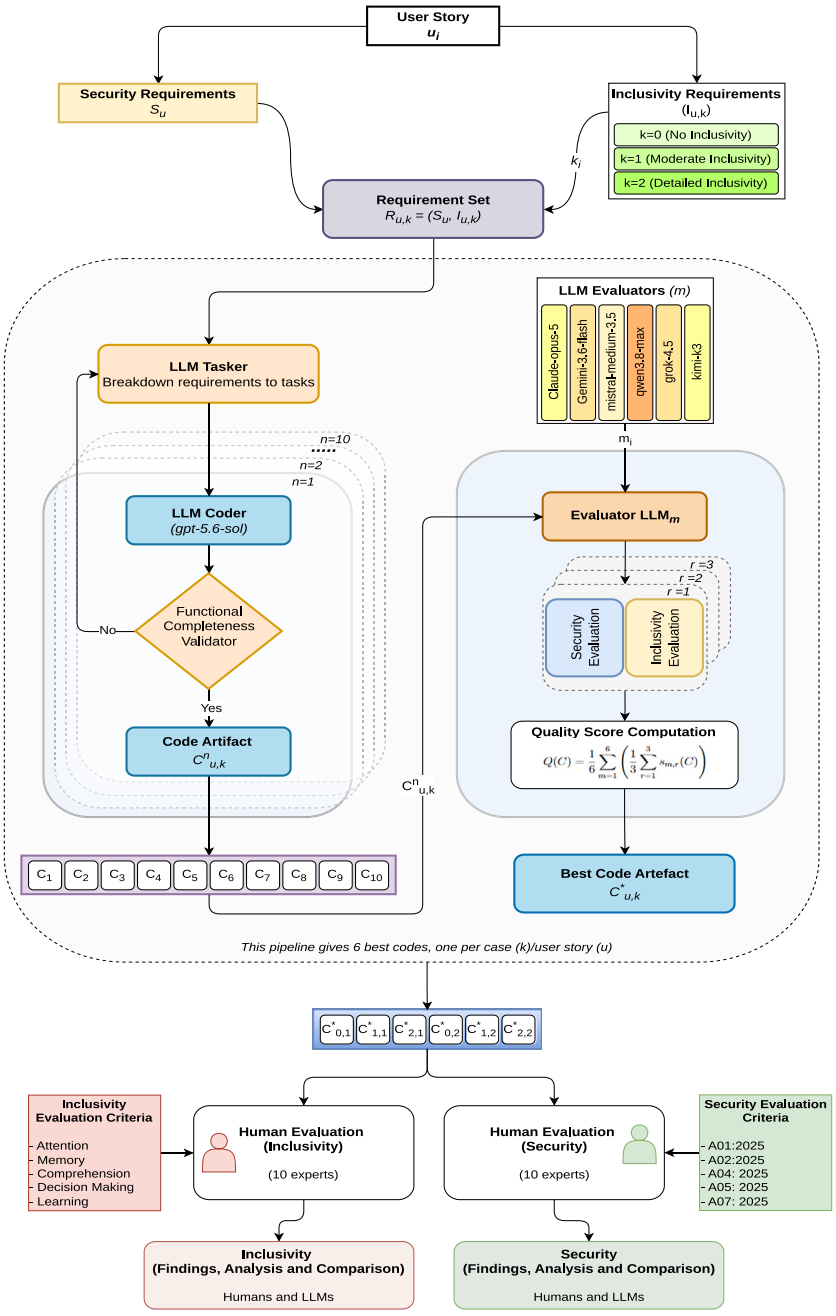


Fig. 1. Experimental setup across two user stories, with three cases varying from minimal to detailed inclusivity specification.

Both user stories feature a user with cognitive challenges in scenarios requiring both security and inclusivity requirements to be appropriately addressed. For this purpose, a fixed set of security requirements was defined for each user story and kep consistent across all cases, aligning with the security needs of password recovery mechanism in *User Story-1* and MFA enrollment in *User Story-2*.

Case 1: No inclusivity specifications

In this case, all references to cognitive conditions were removed from both user stories. The resulting requirement sets contain only functional and security requirements. This case serves as a baseline to observe the default behavior of the LLM in the absence of inclusivity considerations.

3.1.1. Construction of experimental design

As stated earlier, three cases were individually derived from each user story with same security but varying inclusivity requirements. The details of cases across each user story are discussed as follows.

Case 2: Moderate inclusivity specifications

In this case, only the cognitive condition mentioned in the user stories above was named. However, no further details concerning these conditions were added. This case was intended to examine how listing of a cognitive condition in the requirements influences the generated code artifact.

Case 3: Detailed inclusivity specifications

User Story-1	User Story-2
In this case, the user story is enhanced with a detailed description of ADHD-related challenges and explicit inclusivity requirements. The following details, in addition to what was previously specified, were added:	The user story is enhanced with a detailed description of dyslexia-related challenges and explicit inclusivity requirements. The following details, in addition to what was previously specified, were added:
Cognitive condition's description: People with ADHD can lose track of multi-step tasks, get easily distracted, and feel overwhelmed by unclear instructions or time pressure. The login and password reset flow must be forgiving, clear, and low-stress. Avoid dense instructions, timeouts, or unexpected page changes. Support memory and focus by showing visible progress and offering reminders of the next step.	**Cognitive condition's description:** People with dyslexia can find dense text, long character strings, and time-pressured reading difficult. The MFA enrollment and verification flow must reduce reading effort, avoid asking to manually enter long codes, and should not impose any time limits based on the user's reading speed. To better support user understanding, use plain language, clear spacing, and pair text with icons.
Additional requirements: • Keep the process simple and structured with clear, step-by-step guidance. • Use straightforward and consistent language without unnecessary information. • Provide clear feedback at each step so the user knows what has happened and what to do next. • Allow enough time for actions without rushing or session timeouts. • Reduce visual and cognitive distractions in the user interface. • Ensure the user can pause and return to the task without losing progress. • Make help options easy to find and access at any stage.	**Additional requirements:** • Keep a clear and dyslexia friendly typeface with suitable line spacing. • Use short and clear instructions with no dense text. Include a brief example of the expected input (like an email or code). • Avoid any moving or cluttered elements on the interface. • Avoid manual entry of any codes and support copy, QR codes, autofill, and password managers. • Ensure consistency and predictability of all steps and allow the user to retry or re-request codes. • Give the user ample time to read and enter codes and provide clear error messages and how to fix them.

3.2. Formalization of the experiment

Let $u \in \{1,2\}$ represent two user stories where $u = 1$ denote the password recovery story and $u = 2$ denote the MFA enrollment user story. Let $k \in \{0,1,2\}$ represent the three levels of inclusivity requirements, where $k = 0$ corresponds to requirements with no inclusivity specification ($I_{u,0}$), $k = 1$ represents requirements with moderate inclusivity specifications ($I_{u,1}$), and $k = 2$ requirements with detailed inclusivity specifications ($I_{u,2}$). Therefore, k reflects the degree of inclusivity that is introduced to software requirements alongside the fixed set of security requirements S_u, for each user story. So, for each case k, the requirement set is:

$$R_{u,k} = (S_u, I_{u,k})$$

Given the requirement set $R_{u,k}$, the LLM coder uses its code generation function f_{LLM} to produce code artifacts. Since LLMs can produce different outputs every time, the code was run ($N = 10$) times for each requirement set at a fixed temperature, generating 10 independent code artifacts for each case, as shown in Fig. 2(a and b). Each of these runs was conducted independently but under the same conditions.

$$C_{u,k}^{(n)} = f_{\text{LLM}}(S_u, I_{u,k}), \qquad n \in \{1, \ldots, N\}$$

Here, (n) denotes each generation run, and each $C_{u,k}^{(n)}$ is an independent code artifact generated by the LLM for the same requirements. The set of code artifacts generated for user story u for case k is:

$$G_{u,k} = \left\{ C_{u,k}^{(1)}, C_{u,k}^{(2)}, \ldots, C_{u,k}^{(N)} \right\}$$

Running the code generation model multiple times helps in capturing the variability in LLM outputs. The model variability was also seen in practice as the code generator LLM failed to produce a working user interface, once in case ($k = 2$) of User Story 1 and twice in case ($k = 2$) of User Story 2. As these three runs could not be evaluated for inclusivity, they were discarded. Across these two user stories, three cases per story, and ten runs per case produced a total of $2 \times 3 \times 10 = 60$ total code artifacts, of which three were discarded, leaving 57 valid code artifacts. Each of these 57 code artifacts was evaluated by six different LLM evaluators. To account for variability in evaluations, each of the six LLM evaluators evaluated each artifact against the security and inclusivity criteria, three times. The three evaluation scores were averaged to obtain the evaluator's aggregate score for a particular artifact. Moreover, the scores from all six LLM evaluators were averaged to get the overall quality score for the code artifact.

$$Q(C) = \frac{1}{6} \sum_{m=1}^{6} \left(\frac{1}{3} \sum_{r=1}^{3} s_{m,r}(C) \right).$$

Here, $s_{m,r}$ denotes the score assigned to code artifact C by LLM evaluator (m) on iteration $(r^t h)$. For each user story u and case k, the artifact with the highest quality score was retained for further analysis.

$$C_{u,k}^{*} = \arg\max_{C \in G_{u,k}} Q(C).$$

It is important to mention that the quality score of the artifact was calculated as the equally weighted mean of the artifact's inclusivity and security scores averaged across all six LLM evaluators. This weighting reflects the equal importance assigned to security and inclusivity in the quality assessment. Consequently, six code artifacts, one for each case across the two user stories, were retained, as shown in Fig. 2(c and d). These six code artifacts were then evaluated for security and inclusivity by human evaluators, and further evaluated for security by static and dynamic security testing tools.

3.3. LLM-based code generation pipeline

The user stories, $u \in \{1,2\}$ have been used to extract varying levels of inclusivity requirements $I_{u,k}$, whereas the security requirements S_u remain fixed for each user story. The experimental workflow is organized into a modular LLM-based pipeline consisting of three key components: the *LLM Tasker*, *LLM Coder*, and *Functional Completeness Validator*. These modules decompose the requirement set into implementable tasks, generate code artifacts, and assess functional completeness, respectively.

Table 1
Summary of key focus areas and their relation to this study.

Focus areas	Relation to this research	Citations
Security of LLM-generated code	These research works have identified security vulnerabilities in LLM-generated code and established the need to evaluate security in LLM-generated artifacts	Dora et al. (2025), Black et al. (2024), Kharma et al. (2025), Ramírez et al. (2024), Hajipour et al. (2024), Zhao et al. (2025), Andročec (2026)
Human vs. LLM evaluation	These works propose having human involvement in the LLM code generation process to avoid full reliance on the models. They emphasize human oversight to reduce security vulnerabilities and improve code quality.	Pasuksmit et al. (2025), Sergeyuk et al. (2026), González et al. (2025), Takerngsaksiri et al. (2025), Xue and Wang (2025)
Inclusivity in LLM-generated code	These research works show that LLMs can improve and fix simple accessibility issues, but they struggle with complex problems. They can also produce incorrect warnings, accentuating the need for a human review	Aljedaani et al. (2024), Suh et al. (2025), Andruccioli et al. (2025)

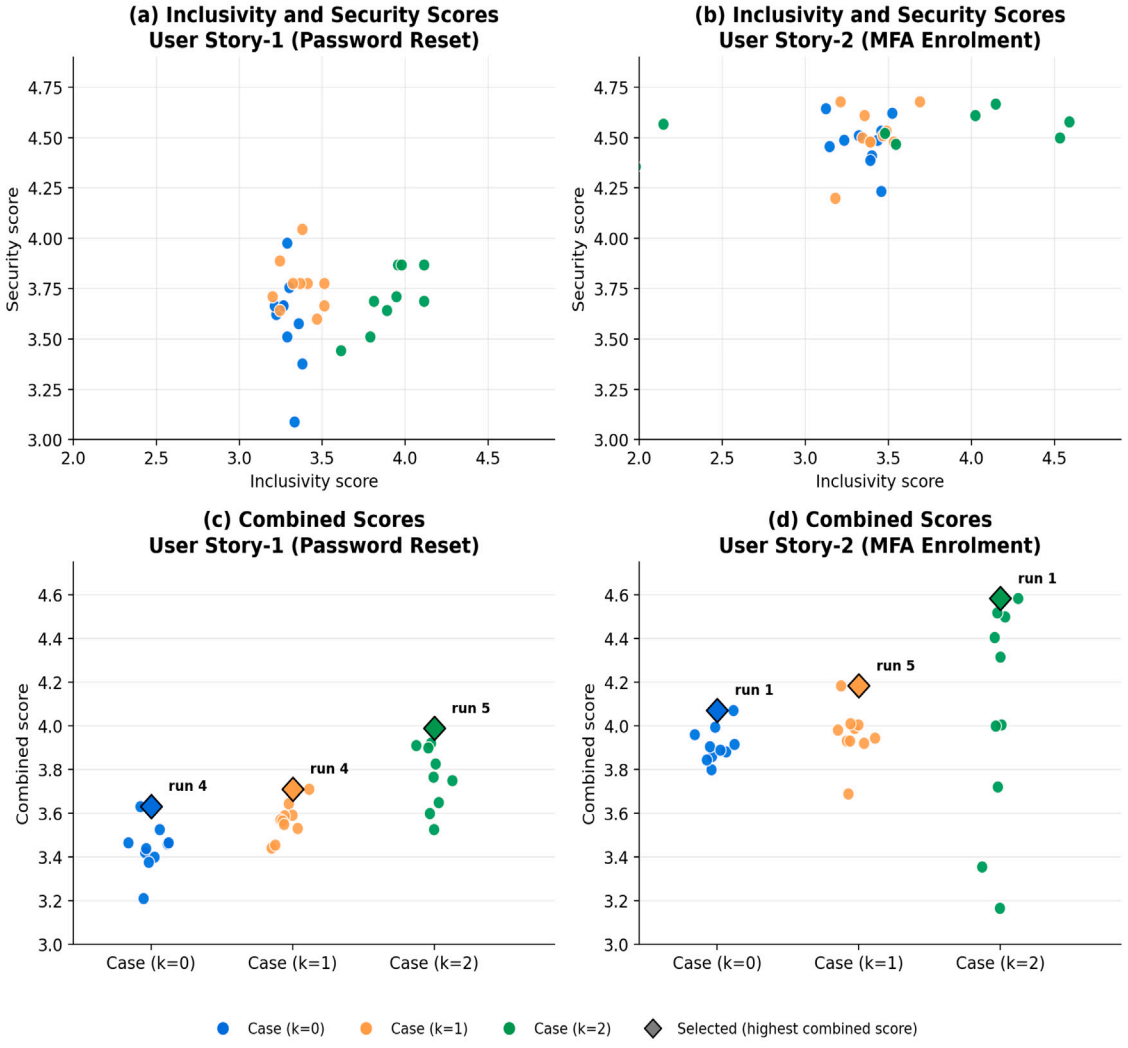


Fig. 2. (a,b) Inclusivity and security scores across six LLM evaluators for successful runs in User Stories 1 and 2. (c,d) Combined scores averaged across all six LLM evaluators for all successful runs in User Stories 1 and 2.

3.3.1. LLM Tasker - requirements decomposition module

The *LLM Tasker* decomposes the composite requirement set $R_{u,k} = (S_u, I_{u,k})$, into a structured list of small, verifiable implementation tasks. It serves as the initialization and coordination module that interprets the requirement set and produces a list of tasks against which code artifacts are to be produced. Algorithm 1 explains how the *LLM Tasker* decomposes the requirement set into tasks. During its first run, when no evaluator feedback or previous task list exists, the *LLM Tasker* extracts atomic tasks directly from $R_{u,k}$ and outputs them as the initial task list. On subsequent invocation, triggered by validator feedback, it cross-references the previous task list T_{prev} and evaluator feedback F to reorganize failed or incomplete tasks and remove those already marked as passed.

The Tasker's output is a strict JSON object containing only one field, ''T'', which enumerates all actionable and non-redundant tasks derived from the current requirement set and validator feedback. This

Algorithm 1 LLM_Tasker (Requirements Decomposition Module)

Input: $R_{u,k} = (S_u, I_{u,k})$, F (optional), T_{prev} (optional)
Output: T (task_list)
$T \leftarrow \emptyset$
$T \leftarrow T \cup \text{ExtractTasks}(R_{u,k})$
if $T_{\text{prev}} = \emptyset$ **and** $F = \emptyset$ **then**
  emit T
  return
end if
if $F \neq \emptyset$ **then**
  $RF \leftarrow \text{ReorganizeFailed}(T_{\text{prev}}, F)$
  $P \leftarrow \text{ExtractPassed}(T_{\text{prev}}, F)$
  $T \leftarrow (T \cup RF) \setminus P$
end if
emit {"T": $[t_1, \ldots, t_L]$}

modular and iterative mechanism ensures that each execution of the pipeline progressively moves toward a functionally complete, secure, and inclusive implementation.

3.3.2. LLM coder - code artifact generation

The GPT-5-based *LLM Coder* implements the ordered task list produced by the *LLM Tasker* against the composite requirements $R_{u,k} = (S_u, I_{u,k})$ to yield a self-contained code artifact. For each case, the *LLM Coder* runs ten (10) times using the task list generated by the *LLM Tasker*. This generates ten code artifacts $C_{u,k}^{(1)}, \ldots, C_{u,k}^{(10)}$. Each run is performed in a separate environment with no shared state that allows capturing the variation in the model's output across all runs. In this study, the term *code artifact* refers to the complete LLM-generated JavaScript source code implementing specified requirements for each case across both user stories. Algorithm 2 shows how the LLM Coder generates each code artifact from the task list and the given requirements. For each case $k \in \{0, 1, 2\}$ across the two user stories $u \in \{1, 2\}$, the same coder was used to generate code $C_{u,k}^{(n)}$.

Algorithm 2 LLM_Coder (Code Artifact Generation)

Input: $R_{u,k} = (S_u, I_{u,k})$ (requirements for case k of story u), T (task_list)
Output: code artifacts $C_{u,k}^{(1)}, \ldots, C_{u,k}^{(10)}$
for $n \leftarrow 1$ **to** 10 **do** ▷ 10 independent runs
  $C \leftarrow \text{BaseScaffold}()$
  for each task $t \in T$ **do**
    $\delta \leftarrow \text{Synthesize}(R_{u,k}, t)$
    $\text{Apply}(C, \delta)$
  end for
  $C_{u,k}^{(n)} \leftarrow C$
  emit $C_{u,k}^{(n)}$ as the full contents of a Code Artifact
end for

Each independent run generates a self-contained code artifact that is ready for downstream functional and quality evaluation. Code artifacts for all cases across both user stories are available in the online repository, as mentioned in Section 8.

3.3.3. Functional Completeness Validator

The *Functional Completeness Validator* verifies whether the generated artifact $C_{u,k}$ fully implements the composite requirements $R_{u,k} = (S_u, I_{u,k})$ and the corresponding task list T produced by the *LLM Tasker*. Given $(R_{u,k}, T, C_{u,k})$, the completeness validator checks each acceptance criterion and each task's intended effect in the artifact. If incompleteness is detected, it synthesizes feedback mapped back to the originating tasks so that the *LLM Tasker* can regenerate a corrected *task list* before code generation continues. The validator emits a structured feedback report and decision object representing PASS/INCOMPLETE as mentioned in Algorithm 3.

Algorithm 3 Functional_Completeness_Validator (Requirements–to–Artifact Completeness Check)

Input: $R_{u,k} = (S_u, I_{u,k})$, T (task list), $C_{u,k}^{(n)}$ (code artifact of run n)
Output: D (PASS / INCOMPLETE), F (feedback)
$F \leftarrow \emptyset$; $D \leftarrow$ PASS
$\mathcal{A} \leftarrow \text{ExtractAcceptanceCriteria}(C_{u,k}^{(n)}, R_{u,k})$
for each criterion $a \in \mathcal{A}$ **do**
  $r \leftarrow \text{EvaluateCriterion}(C_{u,k}^{(n)}, R_{u,k}, a)$
  if r is FAIL **then**
    Append $\text{ConcreteIssue}(C_{u,k}^{(n)}, a)$ to F
    $D \leftarrow$ INCOMPLETE
  end if
end for
emit F ▷ feedback returned to the LLM Tasker when $D =$ INCOMPLETE
emit D

4. Code quality evaluation criteria

The quality of generated code artifacts was evaluated in terms of security and inclusivity by human experts (10 inclusivity experts and 10 security experts) and six LLMs (Claude 5, Gemini 3.6, Mistral 3.5, Kimi K3, Qwen 3.8, and Grok 4.5). The involvement of both humans and LLMs in the evaluation process is in line with $RQ - 2$ listed in Section 1. This human-in-the-loop approach helps not just in identifying potential challenges in secure and inclusive code development but also assists in improving the overall inclusiveness of both password reset and MFA enrollment mechanisms, making them more usable for people with cognitive disabilities.

4.1. Defining inclusivity evaluation criteria

The inclusivity aspect in the conducted experiments focused on five key cognitive dimensions (D_I):

$$D_I = \{\text{Attention}, \text{Memory}, \text{Comprehension}, \text{Decision-Making}, \text{Learning}\}$$

These five cognitive dimensions were carefully chosen as they are recognized as vital cognitive demands for using complex digital systems, including web applications, by cognitive psychology and accessibility guidelines like the Web Content Accessibility Guidelines (WCAG) (Caldwell et al., 2008). These accessibility guidelines aim to help all users focus on relevant information (*Attention*), reduce the need to remember information across different steps (*Memory*), easily understand the system behavior and displayed content on the User Interface (*Comprehension*), avoid errors and make secure decisions in limited time (*Decision making*), and take advantage of the consistent and predictable patterns which can be learned over time (*Learning*). By structuring the study around these five key cognitive dimensions, we link this study of LLM-generated code to established accessibility principles and make these cognitive requirements clear and measurable in secure and inclusive coding tasks. Table 2 shows how user experience across user stories 1 and 2 is affected when these cognitive dimensions are not appropriately addressed.

The LLM-generated codes were evaluated against these five cognitive dimensions. Each dimension $d \in D$ has three evaluation criteria, $q = 1, 2, 3$. Each evaluation criterion is scored on a scale from *1* to *5*. Let $i_{d,q}^{(m,r)}(C_{u,k}^{(n)})$ denote the raw score given by LLM evaluator m for criteria q in dimension d, at repetition r, for code artifact $C_{u,k}^{(n)}$. Here, $C_{u,k}^{(n)}$ represents code artifact produced during run n for case k of user story u. Since each code artifact is evaluated by $(M = 6)$ LLM evaluators,

Table 2
Risks of not addressing cognitive dimensions across both User Stories.

Dimensions	Risks if not addressed	
	User story-1	User story-2
Attention	Helena may lose focus, skip steps, or fail to log in, which delays her access to important health information.	Marcus may get distracted, lose his place during enrollment, or miss the current step, leaving MFA incomplete.
Memory	Helena may forget her credentials or reset code and be unable to access her healthcare account.	Marcus may struggle remembering the OTP or secret codes and without support, he may lose account access.
Comprehension	Helena may misunderstand instructions and enter incorrect information during authentication.	Marcus may find dense and technical instructions difficult to understand and, as a result, enter incorrect information.
Decision-Making	Helena may feel confused, choose the wrong option, or abandon the login process.	Marcus may be unsure which authentication option to choose or what to do next, resulting in selecting the wrong option, ultimately stopping enrollment.
Learning	Helena may struggle to learn the rese t process and needs repeated assistance to access her account.	Marcus may struggle to remember the enrollment steps across sessions and needs repeated help.

repeating their assessment ($R = 3$) times, the final criterion score is calculated by averaging across all evaluators and repetitions, as shown in Eq. (1).

$$i_{dq}(C_{u,k}^{(n)}) = \frac{1}{MR}\sum_{m=1}^{M}\sum_{r=1}^{R} i_{dq}^{(m,r)}(C_{u,k}^{(n)}) \tag{1}$$

For human evaluation, the criterion score $i_{dq}(C_{u,k}^{(n)})$ is calculated by averaging the scores given by ($N = 10$) inclusivity experts, with each expert assessing the code artifact once. The dimension score is therefore computed as the sum of its three aggregated criteria, as shown in Eq. (2).

$$I_d(C_{u,k}^{(n)}) = \sum_{q=1}^{3} i_{dq}(C_{u,k}^{(n)}) \tag{2}$$

The total inclusivity of code $C_{u,k}^{(n)}$ (ranging from *15* to **(75)** is the sum of scores across all five dimensions, as shown in the equation. (3).

$$\mathcal{I}u,k^{(n)} = \sum_{d=1}^{5} I_d(C_{u,k}^{(n)}) = \sum_{d=1}^{5}\sum_{q=1}^{3} i_{dq}(C_{u,k}^{(n)}). \tag{3}$$

The inclusivity evaluation rubrics for User Stories 1 and 2 are given in Appendix A and Appendix C, respectively.

4.2. Defining the security evaluation criteria

This study focuses on an LLM-generated code for password recovery and MFA enrollment mechanisms. Therefore, five OWASP Top 10:2025 categories relevant to these types of flows are chosen (OWASP Foundation, 2021). These categories, denoted as A, include Broken Access Control, Security Misconfiguration, Cryptographic Failures, Injection attacks, and Authentication Failures.

$$A = \{\text{A01:2025}\ ,\ \text{A02:2025}\ ,\ \text{A04:2025}\ ,\ \text{A05:2025}\ ,\ \text{A07:2025}\ \}$$

These attack categories represent the main security vulnerabilities that can be present in flows like password-reset or MFA enrollment. These are not only OWASP labels, but each attack category also matches common weakness types in the MITRE Common Weakness Enumeration (CWE), such as improper access control, improper authentication, injection flaws, configuration errors, and cryptographic misuse (The MITRE Corporation, 2026). Table 3 shows how the selected OWASP categories can impact users with neurodivergent conditions across both user stories.

The LLM-generated codes were evaluated against these five OWASP categories. Each category $a \in A$ has three evaluation criteria, each scored on a scale from 1 (critically insecure) to 5 (most secure). Let $s_{ac}^{(m,r)}C_{u,k}^{(n)}$ represent the raw score given by LLM evaluator m for the cth criterion ($c = 1, 2, 3$) in category $a \in A$, during repetition r. The score is assigned to code artifact $C_{u,k}^{(n)}$, produced in run n for case k of user story u. Since each code artifact is evaluated by six LLM evaluators, with each evaluator repeating it three times, the aggregated criterion score is calculated by averaging the scores across all LLM evaluators and their repetitions, as shown in Eq. (4).

$$s_{ac}(C_{u,k}^{(n)}) = \frac{1}{MR}\sum_{m=1}^{M}\sum_{r=1}^{R} s_{ac}^{(m,r)}(C_{u,k}^{(n)}) \tag{4}$$

For human evaluation, the criterion score $s_{ac}C_{u,k}^{(n)}$ is calculated by averaging the scores given by $N = 10$ human evaluators, with each expert assessing the code artifact once. The security score for each OWASP attack category is calculated in the Eq. (5):

$$S_a(C_{u,k}^{(n)}) = \sum_{c=1}^{3} s_{ac}(C_{u,k}^{(n)}) \tag{5}$$

The total security robustness score $S^{(n)}u,k$ for code artifact $C^{(n)}u,k$ is calculated using Eq. (6).

$$S_{u,k}^{(n)} = \sum_{a\in A} S_a(C_{u,k}^{(n)}) = \sum_{a\in A}\sum_{c=1}^{3} s_{ac}(C_{u,k}^{(n)}) \tag{6}$$

The security evaluation rubrics for User Stories 1 and 2 are given in Appendix B and Appendix D, respectively.

5. Findings

In line with RQ-1, the LLM generated code artifacts were analyzed in terms of both interface and functionality, to evaluate their efficacy from inclusivity and security perspectives. Moreover, for RQ-2, the code artifacts were evaluated from both security and inclusivity perspectives by (i) human experts (ten each for inclusivity and security) and (ii) six LLMs, including Claude 5, Gemini 3.6, Mistral 3.5, Kimi K3, Qwen 3.8, and Grok 4.5. As mentioned earlier, multiple code generation iterations per case were done to minimize stochastic noise or random occurrences, therefore, the evaluation scores from each of these iterations were aggregated to select the best generated artifact, used for reporting the LLM evaluation results and subsequent human evaluations. It is relevant to note that as GPT-5 was used for generating the code artifacts, it was excluded from the LLM evaluation group to minimize self-evaluation contamination. The details of human evaluators who participated in the study are presented in Table 4. Human evaluation was conducted through an online survey on Webropol (Webropol, 2025), which implemented the same evaluation rubrics. The

Table 3
Security risks if OWASP 2025 vulnerabilities are not addressed across both user stories.

Attacks	Risks if not addressed	
	User story-1	User story-2
A01:2025	Attacker gains unauthorized access to Helena's account, exposing credentials and health records.	Attackers can bypass authorization and changes/enrolls MFA on Marcus's account, gaining access to his banking.
A02:2025	Attacker retrieves Helena's reset token and impersonates her.	Exposed OTPs, backup codes, or tokens allow attacker to get Marcus's MFA details and impersonate him.
A04:2025	Helena's credentials and health data become exposed during storage or transmission.	Weak/unencrypted secrets and backup codes allows attacker to capture Marcus's authentication data.
A05:2025	Attacker extracts stored credentials, including Helena's account details.	Unsanitised input could let attackers manipulate queries and access stored credentials.
A07:2025	An unauthorized user guesses or obtains the reset code and accesses Helena's profile and sensitive health records.	Weak or poorly protected verification codes allow attackers to guess Marcus's OTP, complete MFA, and access his banking account.

Table 4
Summary of Human Experts Involved in Evaluating LLM-Generated Codes.

Expertise group	Roles represented	Experience mean (Years)
Inclusivity (10 experts)	Software engineer, Tester, Cybersecurity expert, Human Resource Manager, Academia, HCI researcher	8
Security (10 experts)	Web and full-stack developer, Vulnerability analyst, Cybersecurity engineer, Security and malware researcher, Academia	6.6

participants were individually invited to the survey, and no personally identifiable information (PII) was collected. Human evaluators were provided with both the LLM-generated source code and a web link to the hosted application for reporting their evaluations.

Furthermore, to examine whether the evaluation scores correlate with evaluators' experience, each evaluator's experience was correlated with the mean scores across six artifacts. We calculated a Pearson correlation coefficient, which showed a weak, not statistically significant negative association, $r(8) = -.211, p = .558$, indicating no statistically significant association between evaluators' experience and inclusivity ratings. For security, the correlation coefficient showed a moderate positive association, but this relationship was not statistically significant, $r(8) = 0.482, p = 0.159$.

5.1. Inter-rater reliability and internal consistency

As human evaluations involved subjective judgments, *inter-rater reliability* and *internal consistency* were assessed before aggregating ratings for statistical analysis. The inter-rater agreement was assessed using the Intraclass Correlation Coefficient (ICC) (Shrout and Fleiss, 1979), in accordance with methodological recommendations for multi-rater ordinal assessments. Reliability was assessed using six artifacts with ten evaluators rating each artifact. A two-way random-effects model for absolute agreement, $\mathrm{ICC}(2, k)$, was employed because the same group of evaluators evaluated all six artifacts. Inter-rater reliability was examined at the item level by treating each artifact-item combination as a reliability target, and the subsequent analyses used the evaluators' averaged ratings. It is important to mention that for inclusivity, during the data quality review, one evaluator's responses were found to lack meaningful variation across six artifacts, as they had only used two of the five available scale points across evaluations. In contrast, the other evaluators' ratings had a range of 4 with SD 0.77 – 1.47. The evaluator was recontacted, and the task was clarified before the evaluation was redone; revised responses were used in the computation. Consequently, for inclusivity, inter-rater reliability was assessed across 90 artifact–item combinations (6 artifacts × 15 items), with each combination rated by the ten evaluators, yielding $\mathrm{ICC}(2, 10) = 0.755$, 95% CI $[0.675, 0.822]$, $F(89, 801) = 4.58$, $p < .001$; indicating good agreement in accordance with the commonly accepted interpretation thresholds (Koo and Li, 2016). Similarly, for security, inter-rater reliability was assessed across the 90 artifact–item combinations, each evaluated by ten evaluators. The analysis yielded $\mathrm{ICC}(2, 10) = 0.87$, 95% CI $[0.82, 0.90]$, $F(89, 801) = 8.07$, $p < .001$, indicating good agreement in accordance with the commonly accepted interpretation thresholds (Koo and Li, 2016).

Furthermore, the internal consistency of multi-item evaluation instruments was assessed using Cronbach's alpha for each construct across 15 Likert-scale items (10 evaluators, 6 artifacts). The inclusivity scale demonstrated good internal consistency, with Cronbach's $\alpha = 0.859$. Given that the items represent five distinct inclusivity dimensions, this coefficient was interpreted descriptively rather than as evidence of a single homogeneous construct. Similarly, the security scale exhibited acceptable internal consistency with Cronbach's $\alpha = 0.74$. This is consistent with the security evaluation rubric covering five conceptually distinct OWASP Top 10 security categories (A01:2025, A02:2025, A04:2025, A05:2025, and A07:2025), rather than a single homogeneous construct. This coefficient is interpreted as descriptive evidence of item consistency rather than evidence that the scale represents a single homogeneous construct. The evaluator ratings were aggregated by calculating the mean ratings across the ten evaluators for each artifact and evaluation rubric item. These aggregated ratings were then used to obtain the construct and category-level scores reported in the following section.

5.2. RQ-1: How effectively can LLMs generate code that implements security and inclusivity requirements, particularly for neuro-divergent users?

5.2.1. Case 1 (k=0): No inclusivity specification

Overall, the findings indicate that the LLM generated syntactically correct code in Case 1 across all (n=20) iterations, (n=10) in each user story. In case 1, no inclusivity requirements were specified for either user story. This case therefore represents the baseline implementation against which the impact of progressively more detailed inclusivity specifications in cases 2 and 3 can be examined. The LLM generated a simple layout with broadly consistent visual characteristics, including similar layout, typography, color scheme, form presentation, and button styling (see Fig. 3). The primary differences in both implementations were functional and interaction-related rather than visual, reflecting the different security mechanisms and workflow demands associated with the two user stories.

In user story 1 (healthcare scenario), a relatively linear account-recovery and service-request workflow was generated, enabling the user to submit a service request (i.e., booking an appointment) by first recovering their account by completing security verification, creating a new password, and acknowledging the updated privacy conditions. However, in user story 2 (banking scenario), the generated implementation involved a more complex multi-factor authentication procedure, requiring the user to verify their identity, configure and confirm an authenticator, manage recovery codes, regenerate those codes, and subsequently logging out (see Fig. 4). In addition to the primary difference between the two implementations mentioned earlier, the MFA enrollment steps (in user story 2) were presented as sequentially numbered in the workflow, providing the users with a clear indication of the current step and the actions required to progress through the process.

5.2.2. Case 2 (k=1): Moderate inclusivity specification

In Case 2, the user's cognitive conditions, namely ADHD and dyslexia, respectively, were added to the requirements specification. As in Case 1, the LLM generated syntactically correct code across 20 iterations, with 10 iterations for each user story. Overall, the visual structure of the interface remained consistent with the baseline implementation (Case 1), including the page layout, form structure, and button styling (see Fig. 5).

The workflow in both user stories remained sequential and consistent; however, compared with Case 1, the generated implementation in Case 2 included more explicit guidance and instructions throughout the interaction. For instance, in user story 1, the recovery process clearly communicated that "we will provide the same response whether or not it is associated with an account". Furthermore, a safety reminder was made more explicit in a clearly separated information box, with a different background color.

In user story 2, the initial secure sign-in form was simplified by requiring the user to input only the email address and mobile phone number (see Fig. 6). Furthermore, the implementation also provided explicit information to the user concerning verification code visibility, expiration time, and one-time-user restriction. In addition, a "Start again" option was provided following an unsuccessful verification attempt. In terms of cross-comparison between Cases 1 and 2, the main difference was therefore observed in terms of explicitness of the information and interaction feedback rather than in a substantial change to the visual design or underlying security workflows.

5.2.3. Case 3 (k=2): Detailed inclusivity specification

In case 3, detailed inclusivity requirements were specified, including what each respective cognitive condition entails. The LLM generated syntactically correct code across 17 out of 20 iterations, with exceptions in iteration 10 of user story 1, and iterations 4 and 7 of user story 2, respectively, which did not produce syntactically correct outputs. It is relevant to state that identifying the reasons for this is beyond the scope of current work.

In user story 1, a distinct progress panel was added on the left. In addition, the content sections were clearly separated, with form fields and action buttons available throughout the recovery process (see Fig. 7). The workflow consisted of six steps: starting the recovery request, verifying the recovery code, creating a new password, confirming a security code, accepting the updated privacy conditions, and completing the appointment booking. In comparison with the earlier cases, the implementation provided more explicit feedback and guidance at each stage of the interaction. For instance, the password-reset step included an explicit password checklist specifying minimum length, character requirements, and allowed character types. Across all steps, important instructions were visually emphasized using a distinct background color and clearly separated information boxes.

Furthermore, in User Story 2, a progress indicator was displayed at the top of the interface. The content sections were clearly separated, with form fields, instructional messages, and action buttons provided throughout the MFA setup process (see Fig. 8). The workflow consisted of explicit stages, including signing in to initiate MFA setup, verifying the user's identity, setting up an authenticator, adding and confirming the authenticator, creating and saving recovery codes, and completing the MFA configuration. The implementation provided explicit feedback and guidance at each stage of the interaction. For example, the authenticator setup stage provided users with alternative methods for configuration, including scanning a QR setup pattern or manually entering a setup secret. Across the workflow, instructions and feedback were visually emphasized using distinct background colors, helping to distinguish informational and status messages from the primary interaction elements. For example, the home screen explicitly stated "Sign in to set up MFA", making the primary purpose of the application immediately apparent; this was less explicit in the first two cases.

Overall, the observations across the three cases from both user stories indicate that increasing the level of specification progressively influenced the generated implementations. In Case 1, where no explicit inclusivity requirements were provided, the LLMs generally produced basic implementations based on the functional security requirements of each user story. In Case 2, specifying the cognitive conditions (ADHD and dyslexia) resulted in more explicit guidance and instructions in the workflows; however, the overall visual structure remained largely consistent with Case 1. In Case 3, where the requirements were further specified with more concrete and operational guidance, additional implementation features became evident, including more structured information, progress bars, and more explicit interaction and security feedback. This progression was observed in both user stories, although the specific implementation changes differed according to the requirements and workflow of each task. Furthermore, the generated code produced the desired output in 57 out of 60 iterations, with three iterations producing no output, as reported earlier.

Moreover, in User Story 2 case 3, despite the specification to avoid manual entry of codes while supporting QR codes and copy-based alternatives, the model retained a manual entry setup of the code; a QR-based setup was also provided (see Fig. 8). This suggests that LLM-generated implementations may introduce or retain interaction mechanisms that are not fully consistent with the stated requirements. Effective prompts, therefore, need explicit anti-requirements (e.g., "no animations or motion", "never log tokens or secrets", "no revealing error text") and unambiguous priority language ("treat any secret logging as a security failure").

Summary of Findings for RQ-1

- Specification granularity is the key to both quality and consistency.
- Defining concrete quality criteria is critical, not only in terms of desired inclusivity features, but also in constraining counterproductive design elements.

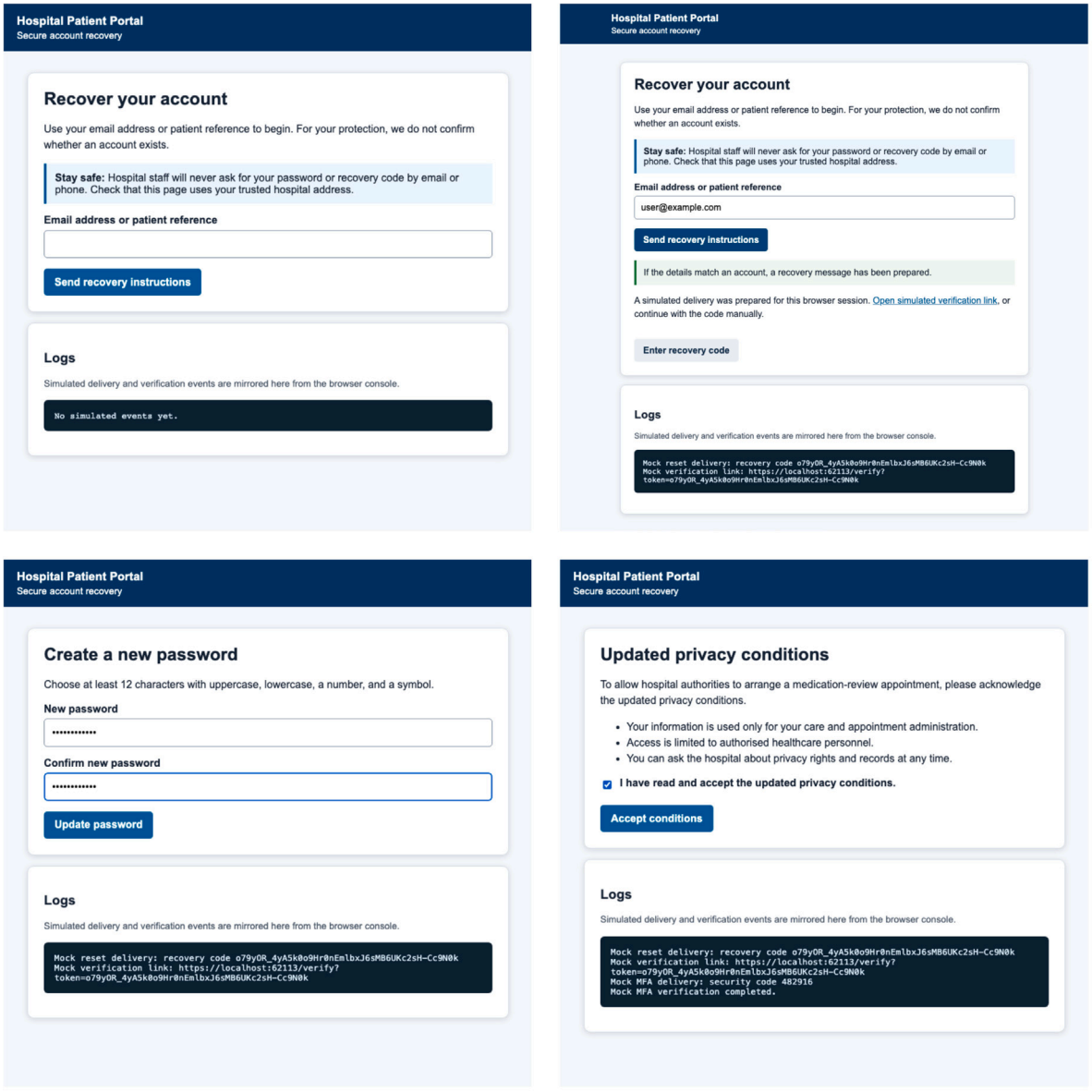


Fig. 3. Screenshots of workflow in case 1 user story 1.

5.3. RQ-2: How does LLM-generated code perform when reviewed by humans and LLMs in terms of security robustness and inclusivity adequacy?

Inline with RQ-2, inclusivity and security evaluations, were performed on the LLM-generated code according to the evaluation criteria outlined earlier. The details are discussed in subsequent sub-sections.

5.3.1. Inclusivity evaluation of the code artifacts

To evaluate the quality of code from an inclusivity perspective, both human and LLM evaluators were tasked with evaluating inclusivity in all three cases across both user stories.

Inclusivity evaluation by human experts. The mean inclusivity scores for three cases across both user stories are presented in Table 5. For user story 1, the table shows that in Case-1 (k=0), the average inclusivity score was 3.87 on a scale of 5. This score indicates an average across all 5 characteristics of the inclusivity rubric. Furthermore, a slight increase in the score to 3.89 was observed in Case 2, which is consistent with the observation in RQ-1 that mentioning the cognitive condition did not change the interface or the workflow. Finally, when detailed specification was provided, the inclusivity score increased to 3.95. For user story 2, the table shows that in Case 1 the inclusivity score was 3.51, which increased to 4.04 and 4.33 in cases 2 and 3, respectively, when the specification was increased. Overall, the results show a consistent increase in inclusivity scores from Case 1 to Case 3 across both user stories, although the magnitude of this increase was different. In User Story 1, the increase was relatively small, from 3.87 in Case 1 to 3.95 in Case 3, whereas in User Story 2 there was an increase from a score of 3.51 in Case 1 to 4.33 in Case 3. Although the increase in User Story 1 was modest, a visible pattern can still be observed across the cases, with higher inclusivity scores corresponding to increasingly detailed specifications.

Table 5
Human evaluation of inclusivity across user stories and cases. Values are reported as mean ± standard deviation.

User story	Case 1	Case 2	Case 3
User Story 1	3.87 ± 1.11	3.89 ± 1.06	3.95 ± 0.98
User Story 2	3.51 ± 1.19	4.04 ± 1.11	4.33 ± 0.82

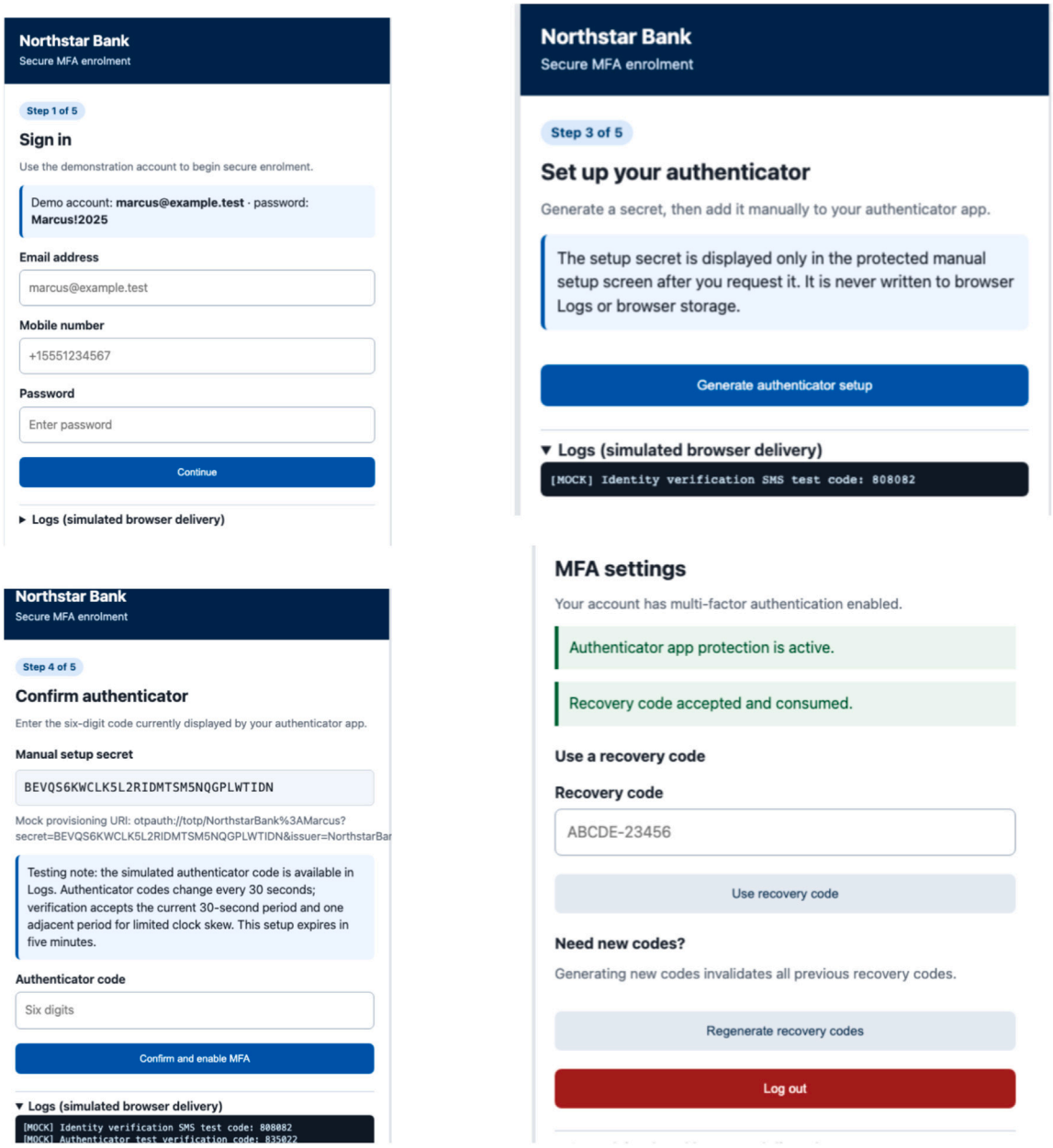


Fig. 4. Screenshots of workflow in case 1 case user story 2.

Furthermore, in addition to the overall score averages across all three cases, the average score for each characteristic in the inclusivity evaluation rubric (across three cases) is presented in Table 6. As indicated in the table, the inclusivity criteria that received the lowest scores were memory and comprehension, respectively. In addition, for attention, decision-making, and learning, the scores did not show substantial variations.

Inclusivity evaluation by LLMs. In line with RQ-2, a panel of six LLMs (mentioned earlier) was used to perform LLM evaluation of the generated artifacts. Each LLM evaluated an artifact three times, and the mean of these three evaluations across the user stories and cases is presented in Table 7. Moreover, the LLMs were instructed to follow the same evaluation rubric as human evaluators, and they were given the same prompts and descriptions.

For user story 1, the table shows that in Case-1 (k=0), the average inclusivity score across all LLMs was 3.31 on a scale of 5. Furthermore, a slight increase in the score to 3.36 was observed in Case 2, which is consistent with the findings reported inline with RQ-1. Finally, in Case 3, the inclusivity score was increased to 4.16. Furthermore, for user story 2, the table shows that in Case 1 the inclusivity score was 3.5, which increased to 3.61 and 4.44 in Cases 2 and 3, respectively, when the specification was increased. Overall, the results show an increase in inclusivity scores from Case 1 to Case 3 across both user stories. In

Table 6
Detailed inclusivity evaluation scores by human evaluators across user stories and cases.

Inclusivity characteristic	User story 1			User story 2		
	Case 1	Case 2	Case 3	Case 1	Case 2	Case 3
Attention	4.63	4.46	4.16	4.07	4.33	4.57
Memory	3.13	3.23	3.67	2.90	3.63	4.37
Comprehension	3.73	3.50	3.63	3.50	3.97	4.07
Decision-making	3.87	4.00	4.03	3.67	4.10	4.23
Learning	4.00	4.27	4.23	3.43	4.17	4.43

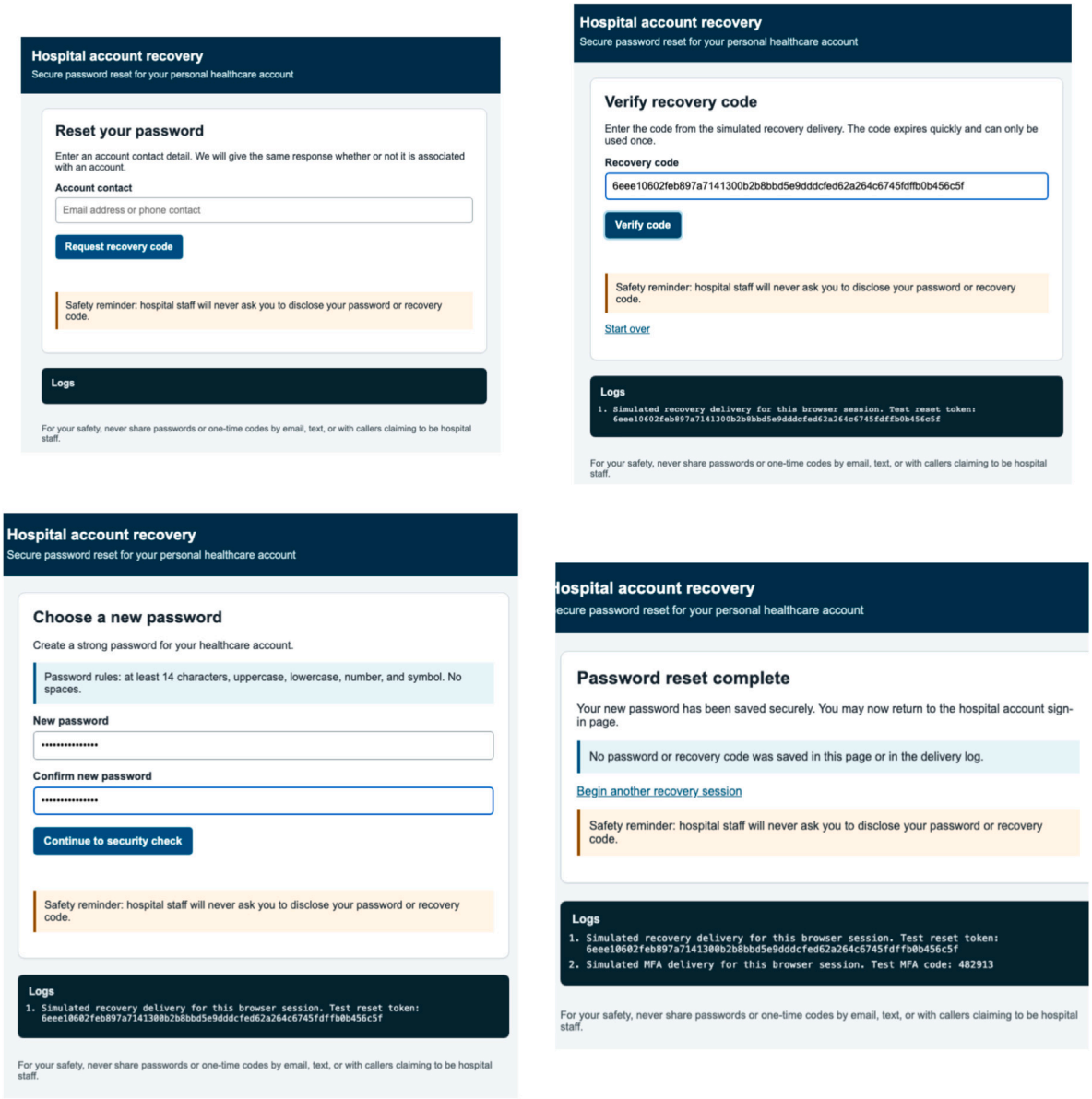


Fig. 5. Screenshots of workflow in case 2, User Story 1.

Table 7
LLM evaluation scores across cases for User Stories 1 and 2. Values are reported as mean ± standard deviation.

LLM	User story 1			User story 2		
	Case 1	Case 2	Case 3	Case 1	Case 2	Case 3
Claude 5.0	3.27 ± 1.01	3.31 ± 1.03	3.60 ± 0.77	3.00 ± 0.84	2.96 ± 0.93	4.24 ± 0.60
Gemini 3.6	3.31 ± 1.16	3.36 ± 1.06	4.27 ± 0.59	3.64 ± 0.95	3.64 ± 0.95	3.80 ± 0.30
Mistral 3.5	3.24 ± 1.06	3.40 ± 1.05	4.53 ± 0.51	3.49 ± 1.43	3.40 ± 1.37	4.73 ± 0.28
Kimi K3	3.29 ± 1.06	3.47 ± 0.90	3.93 ± 0.70	3.56 ± 0.80	3.76 ± 0.90	4.56 ± 0.34
Qwen 3.8	3.44 ± 0.94	3.42 ± 0.81	4.09 ± 0.51	3.42 ± 0.87	3.73 ± 0.99	4.69 ± 0.14
Grok 4.5	3.29 ± 1.18	3.18 ± 1.10	3.93 ± 0.66	3.89 ± 0.89	4.20 ± 0.84	4.62 ± 0.42
Average scores (all models)	3.31 ± 0.98	3.36 ± 0.94	4.16 ± 0.72	3.5 ± 0.97	3.61 ± 1.01	4.44 ± 0.48

terms of individual model scoring, all models gave increasing scores from Case 1 to Case 3 in both user stories. However, the magnitude of increase varied across all models; for instance, the increase was generally more pronounced in Case 3, particularly for Mistral 3.5 and Qwen 3.8, while Claude 5.0 showed a notable increase only in User Story 2. The collective pattern indicates that LLMs converged in their scoring of the inclusivity criteria across the cases.

In addition to the overall evaluation, the evaluation for each inclusivity characteristic, as detailed in the rubric, is presented in Table 8. As indicated in the table, the inclusivity criteria that received the lowest scores were memory and comprehension. The weaker scores suggest that the LLMs consistently perceived deficiencies in features supporting cognitive load reduction and error recovery. In addition, for decision-making and learning, the scores also started low but improved

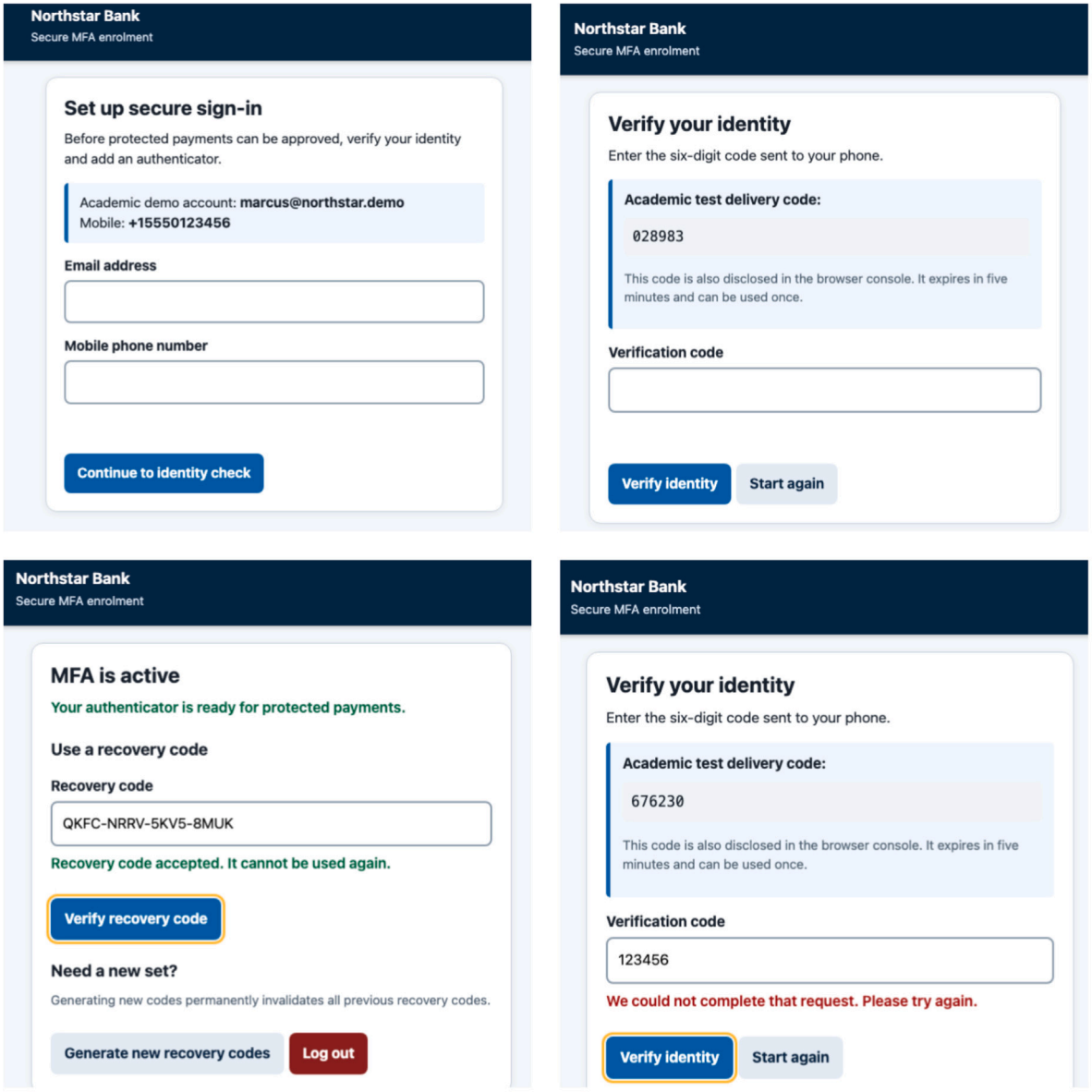


Fig. 6. Screenshots of workflow in case 2, User Story 2.

substantially in the later cases, reflecting enhanced perceived guidance and clarity in the user interface design.

Furthermore, a comparison of the average inclusivity scores for each cognitive dimension by human and LLM evaluators for the two user stories are plotted in Fig. 9. The graphs represent that the agreement among the evaluators varied across some dimensions across both user stories. For example, in User Story 1, the scores for attention, decision-making, and learning were quite similar, however, there was a visible difference between LLMs and humans scores for memory and comprehension, respectively. Similarly, in User Story 2, a visible difference can be seen for memory and comprehension, respectively, while other dimensions received comparable scores.

5.3.2. *Security evaluation of the code artifacts*

The security evaluation was performed in line with the security evaluation criteria discussed earlier.

Security evaluation by human experts. The mean security scores from human evaluators across all three cases for both user stories are presented in Table 9. As shown in the table, a slight decline in security scores was observed for User Story 1 as the level of inclusivity requirements increased, with scores decreasing from 3.61 in Case 1 to 3.39 in Case 3. However, the scores generally remained stable across all three cases in User Story 2. This indicates that increasing inclusivity specification did not have a meaningful impact on the security of the code artifacts.

Table 10 presents the category-level scores by human evaluators across both user stories. As presented in the table, the human evaluators considered *A2:2025 Security Misconfiguration*, a prominent issue in the LLM-generated code as represented by consistently lower scores across the three cases; however, the evaluators were confident that the code is well guarded against *A05:2025 Injection* and *A7:2025 Authentication Failures*.

Security evaluation by LLMs. The security evaluation scores by LLM evaluators are presented in Table 11. As indicated in the table, the scores did not show substantial deviation across the three cases in both user stories. Moreover, in User Story 2, the LLMs evaluators rated security implementations higher than in User Story 1. We reiterate here that the security requirements had remained constant across all three cases, and despite increasing inclusivity requirements from case 1 to 3, no noticeable degradation in perceived security of the code

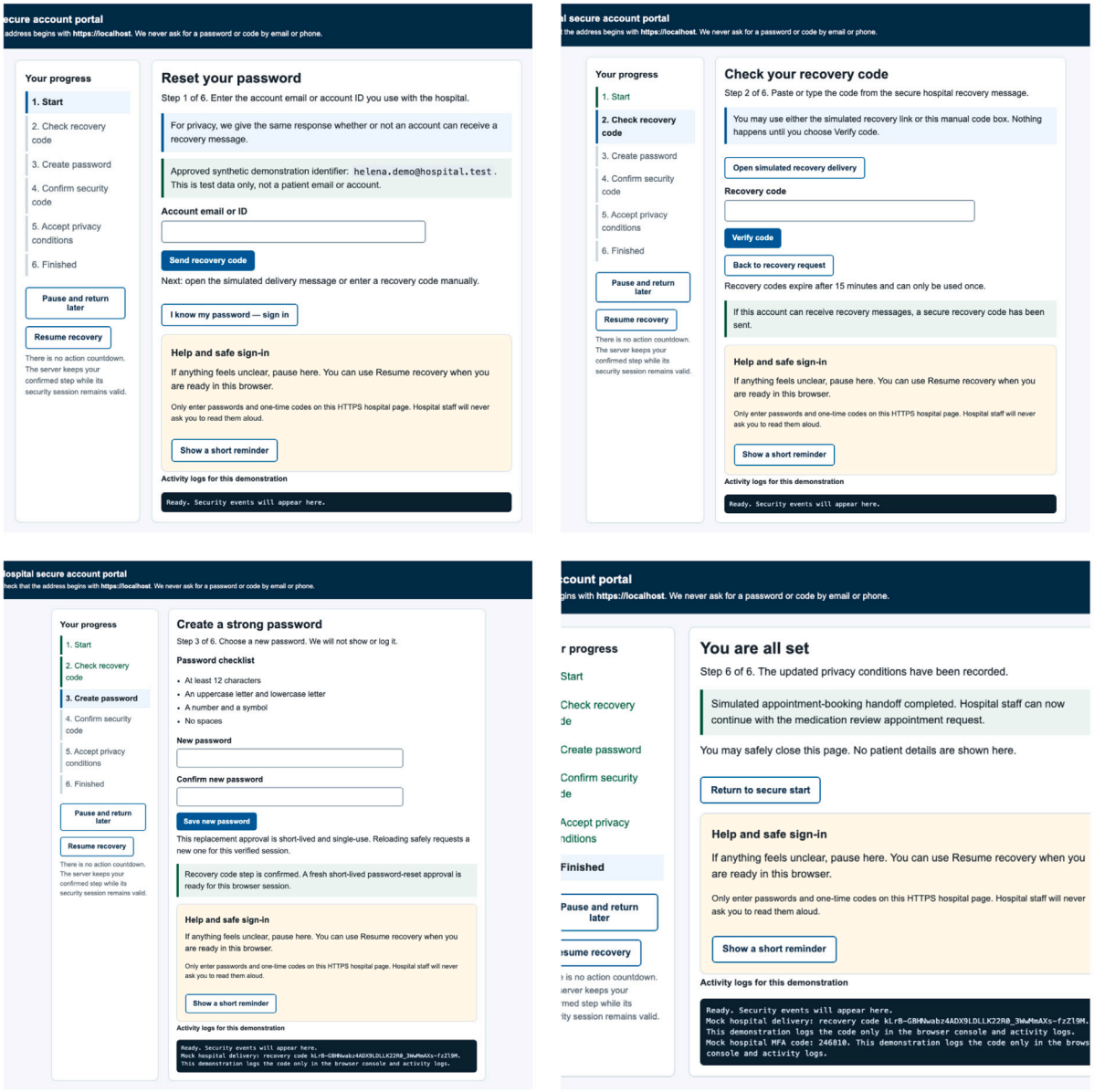


Fig. 7. Screenshots of workflow in case 3, User Story 1.

artifacts was observed. Furthermore, category-level scores across the three cases in both user stories are presented in Table 12. As presented in the table, lower scores were observed in the case of Broken Access Control *A01:2025*, however, higher scores were observed in the case of Cryptographic and Authentication Failures, *A4:2025* and *A7:2025* respectively. In addition, it is relevant to note that *A05:2025 Injection Attacks* received lower scores across the three cases in User Story 1, reflecting the evaluators perception of the implementation being susceptible to injection attacks.

Moreover, in line with RQ-2, a comparison of the category-level mean security scores by human and LLM evaluators across both user stories is presented in Fig. 10 where it can be observed that LLM evaluators generally assigned higher security scores than human evaluators. Moreover, differences in the scores were also observed across OWASP categories, representing varying judgments about perceived security. However, the magnitude of the difference varies across different attack categories, for example, across both user stories, the scoring difference (between the two evaluator groups) is more evident in case of *A02:2025 Security Misconfiguration*. These differences highlight the complementary strengths of human and LLM evaluators and suggest that the two groups should not be treated as interchangeable.

Summary of Findings for RQ-2

- LLM mostly generated syntactically correct code; however, inclusivity concerns were inadequately addressed unless specified explicitly.
- Trade-off situations between security and inclusivity were not observed, as increasing the inclusivity specification did not negatively impact security evaluation scores.
- Hybrid evaluation strategies combining human expertise and LLM-based evaluations may provide complementary strength for evaluating both inclusivity and security aspects of the code.

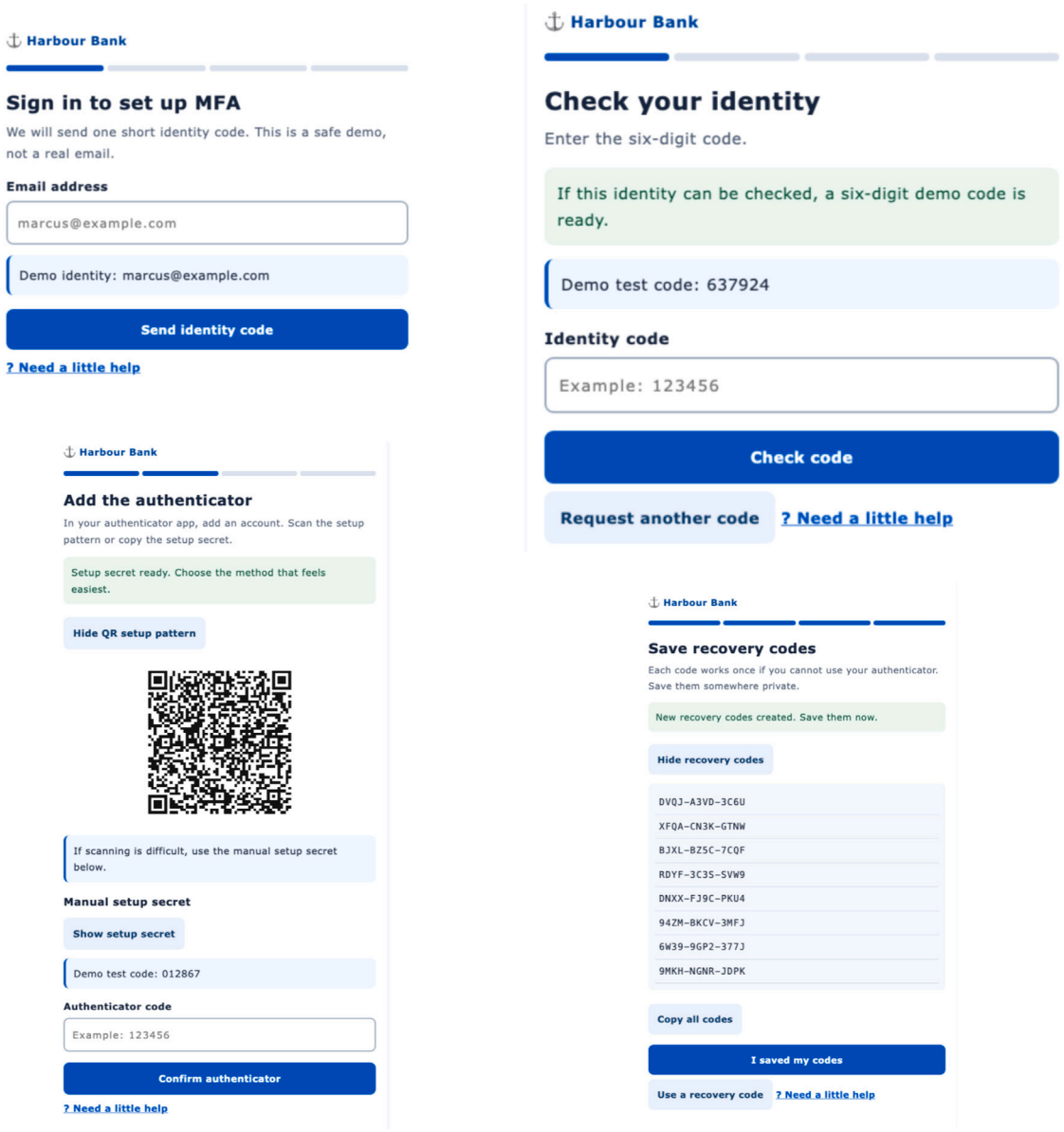


Fig. 8. Screenshot of workflow in case 3, User Story 2.

5.4. Hypothesis evaluation

In addition to the findings presented earlier, Figs. 11 and 12 compare inclusivity and security scores, respectively, across the three cases for each of the two user stories, as evaluated by human and LLM evaluators. To account for repeated evaluation of the same artifacts by human and LLM evaluators, we analyzed human-LLM comparisons using a mixed-effects model, with evaluator group (human vs LLM) as a fixed effect, and evaluator and artifact as random effects.

1. Hypothesis H1 posited that LLMs can generate syntactically correct and secure code, but that neurodiversity concerns would be underrepresented unless explicitly stated. This hypothesis was **supported**. As discussed in Sections 5.2 and 5.3, and illustrated in Fig. 11, inclusivity scores increased from Case-1 ($k = 0$) to Case-3 ($k = 2$) as the level of specification detail increased.
2. Hypothesis H2a posited that there are systematic differences in evaluation between human reviewers and LLMs in identifying inclusivity challenges. This hypothesis was **not supported**. The mixed-effects analysis did not identify a statistically significant difference between human and LLM evaluations of inclusivity, with mean scores of 3.93 and 3.71 (on a five-point scale) for human and LLM-based evaluators, respectively. The estimated difference between human and LLM evaluations was 0.220, with a 95% CI $[-0.076, 0.516]$ and $p = .144$. Therefore, the analysis did not detect a statistically significant difference between the two evaluator groups.
3. Hypothesis H2b posited that there are systematic differences in evaluation between human reviewers and LLMs in evaluating security robustness. This hypothesis was **supported**. The mixed-effects analysis identified a statistically significant difference between the two evaluator groups. LLM evaluators assigned higher security-robustness scores than human evaluators, with mean scores of 4.32 and 3.79, respectively. The estimated difference between LLM and human evaluations was 0.526 on the five-point scale, with a 95% CI $[0.203, 0.849]$, $p = .001$. These results indicate a statistically significant difference between the two evaluator groups in their ratings for the security robustness of the selected artifacts.

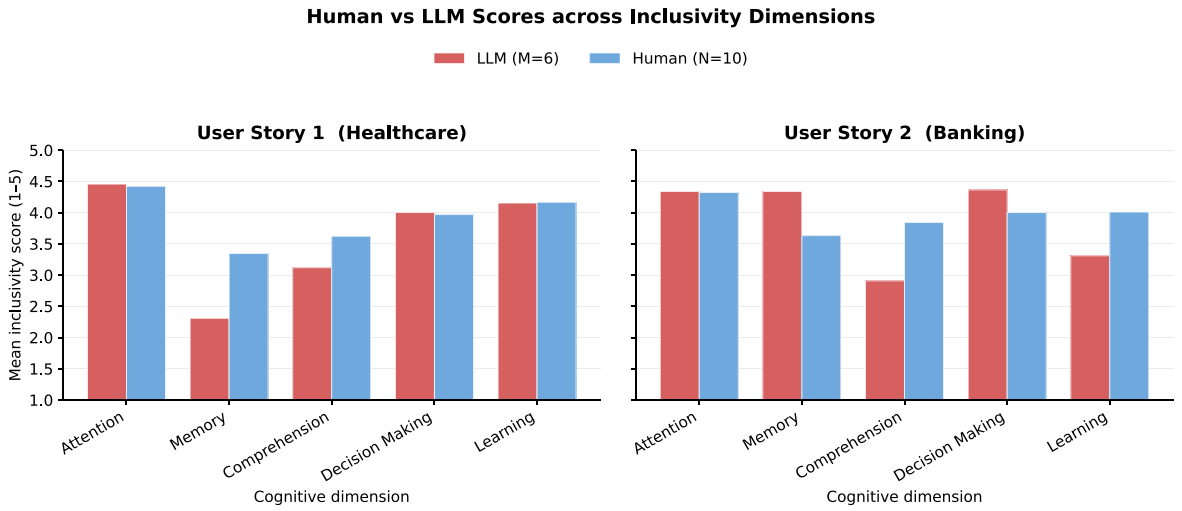


Fig. 9. Cross-comparison of average inclusivity scores by human and LLM evaluators across User Stories 1 and 2.

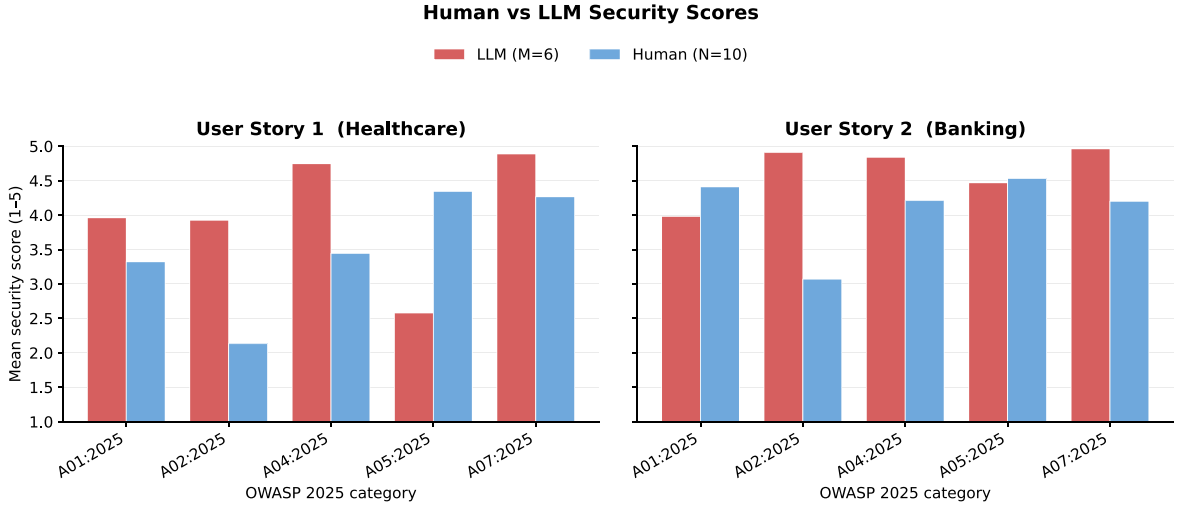


Fig. 10. Cross-comparison of average security scores by human and LLM evaluators across User Stories 1 and 2.

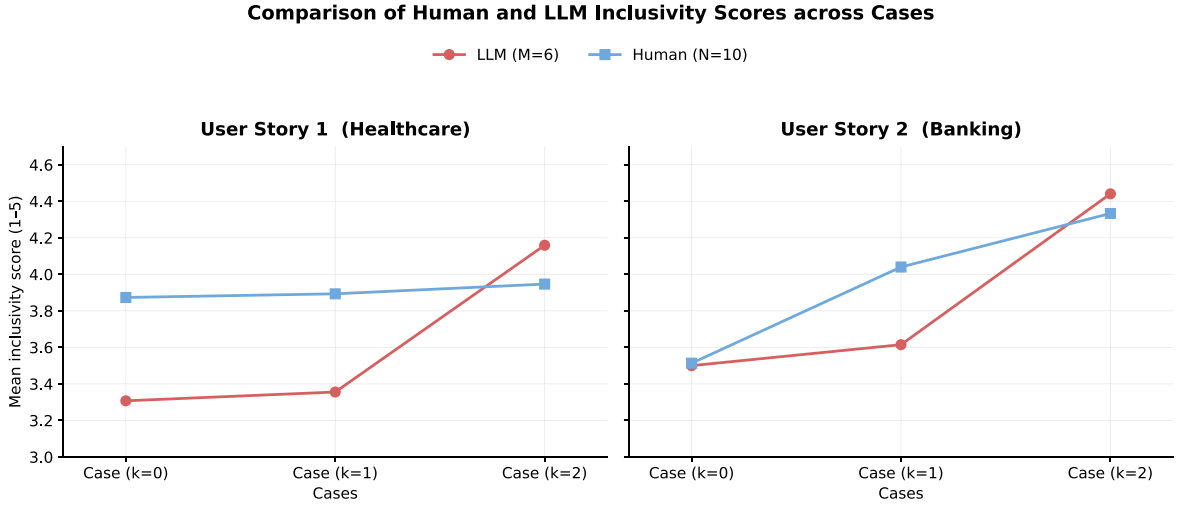


Fig. 11. Cross-comparison of human and LLM inclusivity scores across three cases for User Stories 1 and 2.

Table 8
LLM inclusivity evaluation scores across user stories and cases.

LLM	Criterion	User story 1			User story 2		
		Case 1	Case 2	Case 3	Case 1	Case 2	Case 3
Claude 5.0	Attention	4.11	4.33	4.11	3.44	3.89	4.33
	Memory	1.78	1.78	2.56	3.56	3.67	5.00
	Comprehension	2.67	2.78	3.00	1.89	1.67	4.33
	Decision-making	4.00	3.67	4.11	3.78	3.22	4.22
	Learning	3.78	4.11	4.22	2.33	2.33	3.33
Gemini 3.6	Attention	4.56	4.44	4.33	4.33	4.22	4.00
	Memory	1.78	1.78	3.67	4.33	3.89	4.00
	Comprehension	2.44	2.89	3.67	2.44	2.11	3.67
	Decision-making	3.78	3.56	4.78	4.33	4.56	4.00
	Learning	4.00	4.11	4.89	2.78	3.44	3.33
Mistral 3.5	Attention	4.33	4.33	4.00	4.67	4.44	4.67
	Memory	1.67	1.78	4.00	3.67	3.78	5.00
	Comprehension	3.11	3.22	4.67	1.11	1.00	4.33
	Decision-making	4.11	4.33	5.00	4.56	4.00	5.00
	Learning	3.00	3.33	5.00	3.44	3.78	4.67
Kimi K3	Attention	4.22	4.00	4.33	3.89	4.22	4.67
	Memory	1.67	2.11	2.89	4.11	4.56	5.00
	Comprehension	2.78	3.00	3.56	3.00	2.67	4.33
	Decision-making	3.78	3.89	4.33	4.33	4.44	4.67
	Learning	4.00	4.33	4.56	2.44	2.89	4.11
Qwen 3.8	Attention	4.33	4.33	4.33	3.78	4.33	4.78
	Memory	2.22	2.33	3.44	4.22	4.56	4.78
	Comprehension	2.67	2.89	3.67	2.67	2.44	4.44
	Decision-making	3.89	3.56	4.33	4.11	4.44	4.78
	Learning	4.11	4.00	4.67	2.33	2.89	4.67
Grok 4.5	Attention	4.44	4.44	4.22	4.78	5.00	4.67
	Memory	1.56	1.67	2.89	4.22	4.78	5.00
	Comprehension	2.67	2.67	3.78	2.78	3.00	4.44
	Decision-making	3.67	3.11	4.11	4.56	4.56	5.00
	Learning	4.11	4.00	4.67	3.11	3.67	4.00
Average (all models)	**Attention**	**4.33**	**4.31**	**4.22**	**4.15**	**4.35**	**4.52**
	Memory	**1.78**	**1.91**	**3.24**	**4.02**	**4.20**	**4.80**
	Comprehension	**2.72**	**2.91**	**3.72**	**2.31**	**2.15**	**4.26**
	Decision-making	**3.87**	**3.69**	**4.44**	**4.28**	**4.20**	**4.61**
	Learning	**3.83**	**3.96**	**4.67**	**2.74**	**3.17**	**4.02**

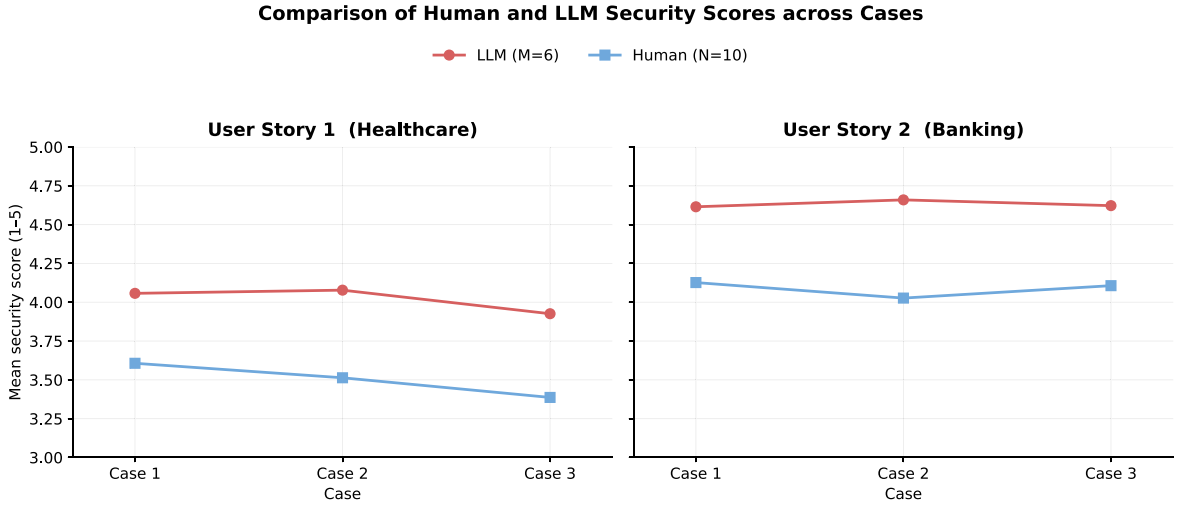


Fig. 12. Cross-comparison of human and LLM security scores across three cases for User Stories 1 and 2.

Table 9
Human evaluation of security across user stories and cases. Values are reported as mean ± standard deviation.

User story	Case 1	Case 2	Case 3
User story 1	3.61 ± 1.36	3.51 ± 1.39	3.39 ± 1.50
User Story 2	4.13 ± 1.27	4.03 ± 1.24	4.11 ± 1.23

Table 10
Detailed security evaluation scores by human evaluators across OWASP categories, user stories, and cases.

OWASP Category	User story 1			User story 2		
	Case 1	Case 2	Case 3	Case 1	Case 2	Case 3
A01:2025	3.47	3.17	3.33	4.53	4.33	4.37
A02:2025	2.37	2.27	1.77	2.80	3.23	3.20
A04:2025	3.50	3.53	3.30	4.13	4.17	4.33
A05:2025	4.33	4.33	4.37	4.73	4.40	4.47
A07:2025	4.37	4.27	4.17	4.43	4.00	4.17

6. Discussion

The results of this study indicate that LLM was generally effective in generating code that addressed the multi-dimensional quality requirements of security and inclusivity. This was observed across two user stories featuring password recovery mechanism in healthcare context and MFA enrollment in online banking context. However, the role of specification is paramount. When the level of specification detail was increased, the code was perceived as more inclusive, while its security remained largely maintained, and no meaningful decline was observed. This pattern remained consistent across the three cases of both user stories.

From an inclusivity perspective, once the LLM coder received more elaborate and structured set of inclusivity requirements, it resulted in outputs with better explanations, simpler wording and language, and a clearly better structure. As mentioned earlier, a notable difference was only observed in case 3 (across both user stories) when detailed set of requirements were provided to the LLM, however, merely naming the inclusivity condition while transitioning from case 1 to case 2 did not result in major improvements. Moreover, no significant difference in inclusivity evaluations was observed between LLM and human evaluator groups.

Furthermore, from a security perspective, results from human and LLM evaluators need to be considered separately, as a significant difference was observed between the two evaluator groups. LLM evaluators rated *A04:2025* and *A07:2025* as high, showing their confidence in the code artifact's defense against cryptographic and authentication failures. However, the human evaluators broadly agreed on code artifacts' performance against authentication failures *A07:2025*, but remained cautious about cryptographic failures *A04:2025*, with low and moderate scores in User Stories 1 and 2, respectively. Such divergence in scoring patterns point towards complimentary strengths that human-in-the-loop can bring to evaluation process, for example, in terms of considering wider contextual issues like key handling and token storage being used in the code. Similarly, both human and LLM evaluators' scores diverged in the case of Injection attacks (*A05:2025*) in User Story 1. This can attributed to the fact that password reset workflows include inputs that are more flexible, like email addresses, tokens, and reset parameters and unsanitised input is more likely to occur in these cases. Moreover, a notable difference between the evaluators' ratings was also observed in the case of Security Misconfiguration *(A02:2025)*, where the human evaluators rated it lower than LLM evaluators across both user stories. Human evaluators ratings can be attributed to deployment and configuration issues like cookie settings, security headers, default credentials, or exposed secrets, which were often not so explicit in the generated code artifact.

These findings clearly indicate that human and LLM assessments are not interchangeable but rather complementary and accentuate the need to combine automated LLM evaluation with human expert review when assessing software quality, particularly from a security point of view.

6.1. Static and dynamic security testing

In addition to the perceived security evaluation performed by the human and LLM evaluators, the assessment was complemented by performing Static Application Security Testing *(SAST)* and Dynamic Application Security Testing *(DAST)* of the six code artifacts representing each case across both user stories. This provided an independent, tool-based assessment of the security of the generated code.

SAST was conducted using two popular and widely used open source tools *Semgrep* v1.177.0 (Semgrep, 2026) and *CodeQL* v2.27.0 (GitHub, 2026). Both tools found no issues in any case of User Story 1, however, in User Story 2, security issues relevant to cryptographic failures *(A04:2025)* and authentication failures *(A07:2025)* were identified (see Table 13). *Semgrep* reported a Cross-Origin Resource Sharing (CORS) origin-reflection pattern (CWE-346), attributable to *(A07:2025)* across all three artifacts of User Story 2. It also found an AES-GCM decryption call without an explicit authentication-tag length (CWE-310), mapped to *(A04:2025)* in case 3 ($k = 2$) of User Story 2. *CodeQL* identified modulo reduction of cryptographically secure random values (CWE-327), mapped to *(A04:2025)* in cases 1 ($k = 0$) and 3 ($k = 2$) of User Story 2, which introduces a small statistical bias in the generated authentication codes.

Moreover, DAST was also performed, where all six selected artifacts were run over *HTTPs* and scanned using *OWASP ZAP* v2.17.0, a widely used open-source dynamic application security testing tool. All scans were on locally running applications rather than any hosted deployment. The findings of *OWASP ZAP* were then mapped to the five OWASP security categories being considered in this work. Informational alerts generated by *OWASP ZAP* as well as alerts related to unrelated third party origins were excluded from the analysis. Across the five OWASP security categories, *OWASP ZAP* only found issues in security misconfiguration *(A02:2025)* as it identified permissive Content Security Policy directives (`unsafe-inline`) as medium risk in cases 1 ($k = 0$) and 2 ($k = 1$) in User Story 2. There were no other DAST findings for the other four OWASP security categories. A summary of the SAST and DAST findings is listed in Table 13.

As indicated in Table 13, SAST and DAST tools found weaknesses in User Story 2 suggesting that the rubric based evaluation may miss some implementation issues which can be identified by analyzing the code. Overall, the four evaluation approaches provide complementary perspectives on the security of code artifacts. It is important to mention that the artifacts (in both user stories) required signing in to access the main functionality, however the scans were run without signing in. The lack of findings relevant to other categories might refer to the situation that no issues were found in the parts of the code accessible without authentication.

6.2. Threats to validity

The potential threats to the validity of the study and mitigation strategies are as follows:

1. Internal Validity: The study was designed to ensure that all evaluators assess the same artifacts using identical evaluation rubrics; however, the subjective interpretation of the rubric item may induce evaluator bias. To mitigate this bias, multiple evaluators were used for the evaluations. Moreover, Inter-rater agreement using the Intraclass Correlation Coefficient (ICC) was computed by treating each artifact-item combination as a reliability target. The result was interpreted cautiously, as the 90

Table 11
LLM security evaluation scores across user stories and cases. Values are reported as mean ± standard deviation.

LLM	User story 1			User story 2		
	Case 1	Case 2	Case 3	Case 1	Case 2	Case 3
Claude 5.0	3.78 ± 1.10	3.64 ± 1.08	3.78 ± 1.11	4.40 ± 0.46	4.47 ± 0.45	4.49 ± 0.43
Gemini 3.6	4.29 ± 1.03	4.38 ± 0.87	3.95 ± 1.50	4.73 ± 0.60	4.73 ± 0.60	4.73 ± 0.40
Mistral 3.5	5.00 ± 0.00	5.00 ± 0.00	5.00 ± 0.00	5.00 ± 0.00	5.00 ± 0.00	4.96 ± 0.10
Kimi K3	4.07 ± 0.91	4.07 ± 1.03	3.86 ± 1.15	4.60 ± 0.57	4.64 ± 0.40	4.62 ± 0.56
Qwen 3.8	3.24 ± 1.15	3.33 ± 1.36	3.09 ± 1.43	4.31 ± 0.55	4.40 ± 0.60	4.40 ± 0.63
Grok 4.5	3.98 ± 1.19	4.04 ± 0.99	3.87 ± 1.37	4.64 ± 0.58	4.71 ± 0.59	4.53 ± 0.65
Average (all models)	**4.06 ± 1.05**	**4.08 ± 1.05**	**3.92 ± 1.24**	**4.61 ± 0.52**	**4.66 ± 0.50**	**4.62 ± 0.52**

Table 12
Detailed LLM security evaluation scores across OWASP categories, user stories, and cases.

LLM	OWASP Category	User story 1			User story 2		
		Case 1	Case 2	Case 3	Case 1	Case 2	Case 3
Claude 5.0	A01:2025	3.89	3.78	3.78	3.78	4.00	4.00
	A02:2025	3.22	3.33	3.67	4.78	4.67	4.78
	A04:2025	4.55	4.22	4.67	4.44	4.67	4.56
	A05:2025	2.22	2.00	2.00	4.11	4.00	4.11
	A07:2025	5.00	4.89	4.78	4.89	5.00	5.00
Gemini 3.6	A01:2025	3.66	3.67	3.67	3.67	3.67	4.11
	A02:2025	5.00	5.00	4.67	5.00	5.00	5.00
	A04:2025	5.00	5.00	5.00	5.00	5.00	5.00
	A05:2025	2.77	3.22	1.44	5.00	5.00	4.56
	A07:2025	5.00	5.00	5.00	5.00	5.00	5.00
Mistral 3.5	A01:2025	5.00	5.00	5.00	5.00	5.00	5.00
	A02:2025	5.00	5.00	5.00	5.00	5.00	5.00
	A04:2025	5.00	5.00	5.00	5.00	5.00	5.00
	A05:2025	5.00	5.00	5.00	5.00	5.00	5.00
	A07:2025	5.00	5.00	5.00	5.00	5.00	4.78
Kimi K3	A01:2025	3.89	4.11	3.78	3.67	4.11	3.67
	A02:2025	4.00	3.89	3.89	5.00	5.00	5.00
	A04:2025	4.89	4.89	4.89	4.89	4.89	5.00
	A05:2025	2.67	2.44	2.00	4.44	4.22	4.56
	A07:2025	4.89	5.00	4.78	5.00	5.00	4.89
Qwen 3.8	A01:2025	3.66	3.89	3.56	3.67	3.67	3.67
	A02:2025	2.44	2.44	2.11	4.78	4.56	4.78
	A04:2025	4.00	4.11	4.22	4.56	4.89	4.77
	A05:2025	1.66	1.44	1.11	3.78	3.89	3.78
	A07:2025	4.44	4.78	4.44	4.78	5.00	5.00
Grok 4.5	A01:2025	3.66	3.67	3.67	3.67	3.67	3.67
	A02:2025	4.11	3.89	4.00	5.00	5.00	5.00
	A04:2025	5.00	5.00	5.00	4.56	4.89	5.00
	A05:2025	2.11	2.67	1.67	5.00	5.00	4.00
	A07:2025	5.00	5.00	5.00	5.00	5.00	5.00
Average (all models)	**A01:2025**	**3.96**	**4.02**	**3.91**	**3.91**	**4.02**	**4.02**
	A02:2025	**3.96**	**3.93**	**3.89**	**4.93**	**4.87**	**4.93**
	A04:2025	**4.74**	**4.70**	**4.80**	**4.74**	**4.89**	**4.89**
	A05:2025	**2.74**	**2.80**	**2.20**	**4.56**	**4.52**	**4.33**
	A07:2025	**4.89**	**4.94**	**4.83**	**4.94**	**5.00**	**4.94**

Table 13
Summary of SAST and DAST findings across the six artifacts.

Tool	Finding (s)	OWASP category	Severity	Cases (US-2)
Semgrep	CORS origin-reflection	A07:2025	Medium	$k = 0, 1, 2$
Semgrep	AES-GCM missing tag length	A04:2025	High	$k = 2$
CodeQL	Modulo bias on secure random	A04:2025	High	$k = 0, 2$
OWASP ZAP	Permissive CSP (`unsafe-inline`)	A02:2025	Medium	$k = 0, 1$

artifact-item combinations (15 items across six artifacts) represent repeated evaluations of the same six underlying artifacts rather than 90 independent artifacts. Another potential threat to internal validity emerges from comparing human and LLM evaluations. As the human evaluations had varying experience, their judgments could be influenced by how they apply and interpret the evaluation criteria. To investigate this, we correlated evaluators' years of experience with their mean scores across the six artifacts. For inclusivity, the analysis showed a weak negative and non-significant association; however, for security, a moderate positive but non-significant association was observed. Therefore, the findings presented in this article do not provide statistically significant evidence that evaluator experience systematically influenced inclusivity or security ratings. Nevertheless, differences in expertise and rating calibration remain possible alternative explanations for variation between human and LLM evaluations and should be considered when interpreting the findings.

2. Construct Validity: In this study, both inclusivity and security constructs were operationalized using a Likert-scale rubric developed based on accessibility guidelines, such as WCAG, and security recommendations like OWASP Top 10. However, the study employed only a subset of these sources (with five relevant evaluation criteria each for inclusivity and security, respectively), rather than a complete set of all possible evaluation criteria. Therefore, to strengthen the reliability of the measurement instruments, internal consistency was assessed using Cronbach's α, which indicated strong internal reliability for both instruments. Nevertheless, the inclusion of other guidelines and recommendations or additional criteria could produce potentially different assessments. In addition, from a security perspective, both Static Application Security Testing (SAST) and Dynamic Application Security Testing (DAST) were conducted on the generated artifacts to objectively demonstrate vulnerability status and to minimize the notion of perceived security. SAST and DAST analyses did not identify major issues, as discussed earlier. However, it is important to consider the inherent coverage and detection limitations of SAST and DAST tools, and the results should be not considered as exhaustive evidence of the absence of vulnerabilities.
3. External Validity: The study evaluated artifacts generated across two user stories, with three cases per user story and ten independent generations for each case. The artifacts were implemented in a particular programming language and featured a particular cognitive disability per user story; therefore, the findings may not directly generalize to other programming languages, development contexts, and use cases. Although repeated generations were conducted to examine variation across cases, the number of underlying user stories and cases remains limited. Nevertheless, future studies involving additional artifacts, diverse development contexts involving other user stories, and a broader set of models are necessary to further validate and generalize the findings.
4. Conclusion Validity: The study involved statistical analyses to compare evaluation scores from human and LLM evaluators. As the same evaluators assessed multiple artifacts, human-LLM comparisons were conducted using mixed-effects models, with evaluator group included as a fixed effect and evaluator and artifact included as random effects. This approach accounts for the repeated evaluations and the shared artifacts in the analysis. However, the study involved a limited number of underlying artifacts, and the selected artifacts were identified through LLM-based evaluation before human evaluation; the statistical findings should be interpreted within the scope of this study.

6.3. Limitations

The study was planned and executed in line with best practices for conducting exploratory case studies; however, there could still be some limitations. First, the human evaluation involved ten evaluators each for inclusivity and security. Nevertheless, a larger and more diverse number of evaluators may provide additional perspectives. Second, six LLMs that represent a subset of currently available LLMs were used for evaluations in the study, which might have induced model selection bias, as newer or differently tuned models might behave differently. Third, as mentioned while discussing construct validity, the evaluation relied on a subset of evaluation criteria based on WCAG and OWASP Top 10 for inclusivity and security, respectively, which may not have captured all possible dimensions of evaluation. Moreover, the experimental design and evaluation instruments are described in sufficient detail to support replication and extension in future empirical software engineering studies. In addition, the study (due to its scope) did not consider inter-LLM differences. As presented in Section 5, all six LLMs differed considerably in their security scores, for example, Mistral Medium 3.5 gave consistently higher scores, while Qwen3.8-Max gave lower scores; this is something that may provide useful insights, and should therefore, be investigated. Lastly, although no substantial decline in security scores was observed at the characteristic level when the inclusivity specification was increased from Case 1 to Case 3; however, at the sub-characteristic/category level, a few observations suggest a potential impact. For instance, a decline in A02:2025 was observed in User Story 1 from Case 1 to Case 3, which indicates a potential security-inclusivity "trade-off" situation. This may have been masked when examined at the higher characteristic-level (security, inclusivity in this case). Therefore, further investigation is required to identify the conditions under which increasing inclusivity may affect specific security aspects at the sub-characteristic/category-level. Such investigation could also provide a stronger theoretical basis for understanding when and how conflicts and trade-off situations may emerge in secure software engineering.

7. Conclusion and future work

The paper presented a study evaluating security and inclusivity in LLM-generated code. The study involved specifications originating from two user stories, each featuring a cognitively challenged user. The specification were progressively detailed in terms of inclusivity requirements from Case 1 to Case 3 for each user story. The results demonstrate the critical role of specification details in determining the quality (security and inclusivity) of LLM-generated code. Moreover, when the LLM-generated code was reviewed by human- and LLM-based evaluators, a significant difference among the evaluator groups for security evaluations was observed, however, no significant difference in the evaluations was observed for inclusivity.

These findings have important research and practical implications for secure software engineering and human-centered AI. From a research perspective, future code-generation models could consider inclusivity-by-default, rather than as an optional feature. Moreover, no substantial degradation of security was observed as the inclusivity requirements were increased. Future research can explore prompt engineering and template design to improve both aspects together. From a practical perspective, there are implications for practitioners in terms of importance of providing clear and detailed instructions when using LLMs for code generation. In addition, since humans and LLMs show complementary strengths, combining both in a hybrid evaluation approach, along with static and dynamic testing, may be more effective for assessing multidimensional quality goals such as security and inclusivity.

The future work may continue in multiple dimensions. First, extending the scope of evaluations beyond the current characteristics, to consider code compliance with guidelines such as ACM SIGSOFT

Table A.1
Inclusivity evaluation rubric for Password Reset Code function.
(1 = poor, 5 = excellent).

Dimension	Evaluation criteria	Score				
		1	2	3	4	5
Attention	User interface is clean and does not have moving images or pop-ups.					
	Important items clearly stand out.					
	It is easy to find out on the screen what she needs quickly.					
Memory	The user does not need to remember many rules or steps.					
	Options like Password manager, autofill, or "remember me" are present.					
	The user can easily recognize saved or familiar items.					
Comprehension	Text is short, clear, and easy to read.					
	Error messages describe the problem and the way to fix it.					
	Examples show what to type or how to do a step.					
Decision-Making	Choices (like password, code, or face login) are clear.					
	The user is not provided with too many choices at one time.					
	The system clearly informs Helena the outcome of each action.					
Learning	The user gets required help or short tips for new steps.					
	The user can easily try again if she makes a mistake.					
	The design stays consistent so that the user can learn it over time.					

guidelines (Ralph and Robbes, 2020). This is particularly important because, unlike traditional Integrated Development Environments (IDEs) that follow consistent coding standards, LLM-generated code varies significantly depending on the model and its version. Second, considering other user stories and developmental contexts, including broader use cases featuring diverse critical and non-critical scenarios. Third, considering quality assessment beyond security and inclusivity to evaluate other important characteristics such as compliance, maintainability, readability, usability, and efficiency in LLM-generated code. Finally, inter-LLM differences will be examined in more details to assess the factors (such as model architecture, training data, alignment strategies, etc.) relevant to these differences.

8. Replication package

To facilitate reproducibility and transparency, we provide a complete replication package accompanying this study (Japskua, 2026), which includes all prompts and input specifications used in the experiment, security and inclusivity evaluation rubrics and scoring criteria, complete human evaluation data, and all generated code artifacts produced across all three cases. The study was initially conducted between September and October 2025, later revised and re-conducted in August 2026. The provided package enables replication of the experimental setup and further extension of the study under different configurations.

CRediT authorship contribution statement

Bilal Naqvi: Writing – review & editing, Writing – original draft, Project administration, Methodology, Investigation, Conceptualization. **Waleed Shahid:** Writing – original draft, Visualization, Validation, Data curation, Conceptualization. **Janne Parkkila:** Writing – review & editing, Writing – original draft, Software, Conceptualization. **Hammad Afzal:** Writing – review & editing, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Inclusivity evaluation rubric for user story-1

See Table A.1.

Appendix B. Security evaluation rubric for user story-1

See Table B.1.

Appendix C. Inclusivity evaluation rubric for user story-2

See Table C.1.

Appendix D. Security evaluation rubric for user story-2

See Table D.1.

Data availability

The link to data is enclosed in the manuscript.

Table B.1
Security Evaluation Rubric for Password Reset Code Function based on OWASP Top 10, 2025.
(1 = Critically insecure, 5 = Most Secure).

Security attack	Evaluation criteria	Score				
		1	2	3	4	5
A01:2025	The system checks that the reset code belongs to the user.					
	The same reset code works for any other user.					
	The previous sessions still work after a password reset by the user.					
A02:2025	The user's private information or tokens are shown in error pages or logs.					
	The passwords and secret codes are stored safely and not in the code.					
	The system allows default credentials.					
A04:2025	The website uses HTTPS with the latest TLS version?					
	The reset codes and user data stored are using an appropriate encryption algorithm.					
	The email message guards/hides the user's private data.					
A05:2025	The code sanitizes user-supplied data like Email and password.					
	The redirect links are safe and go only to the hospital's site.					
	The email templates are safe and contain no script or HTML from user text.					
A07:2025	The system bars the user from making several wrong attempts.					
	The application shows if the username/email does not exist.					
	The password reset code is random, complex, and user-specific.					

Table C.1
Inclusivity evaluation rubric for MFA enrollment Code function.
(1 = poor, 5 = excellent).

Dimension	Evaluation criteria	Score				
		1	2	3	4	5
Attention	The user interface is clean and does not have moving images or pop-ups.					
	Important items clearly stand out.					
	It is easy for the user to find what they need to do on the screen.					
Memory	The user does not need to remember many rules or steps.					
	The system provides options such as copy-to-clipboard or QR codes so the user does not have to manually enter long codes.					
	The user can easily recognize the information and options needed to complete each step.					
Comprehension	Text is short, clear, and easy to read.					
	Error messages describe the problem and the way to fix it.					
	Examples show what to type or how to do a step.					
Decision-Making	Choices (like password, code, or face login) are clear.					
	The user is not provided with too many choices at one time.					
	The system clearly informs the outcome of each action.					
Learning	The user gets the required help or short tips for new steps.					
	The user can easily try again if they make a mistake.					
	The design stays consistent so that the user can learn it over time.					

Table D.1
Security Evaluation Rubric for MFA enrollment Code Function based on OWASP Top 10, 2025. (1 = Critically insecure, 5 = Most Secure).

Security attack	Evaluation criteria	Score				
		1	2	3	4	5
A01:2025	The system verifies that the MFA setup or code belongs to the account of the logged in user.					
	A verification or backup code cannot access any other user's account and only works for its own account.					
	MFA and backup codes can only be set up through the bank's own pages, not through hidden requests from another website.					
A02:2025	Secret keys, one time codes, backup codes, and login tokens never appear in logs, links, or error pages.					
	The site uses secure browser settings, such as secure cookies and protections against framing or reuse by other sites.					
	The app only accepts requests from trusted sites, has no default passwords and hides technical error details.					
A04:2025	The website always uses HTTPS with a secure, up to date connection.					
	The system randomly generates and securely stores MFA secrets and backup codes using hashing or encryption.					
	Login tokens and secret codes are not stored in the phone browser where they could be accessed.					
A05:2025	The app sanitizes user input, including emails, phone numbers, and codes, before using it.					
	Any links provided by the application keep users within the bank's own site.					
	User text is safely displayed and cannot run hidden scripts or code.					
A07:2025	The system blocks further attempts by the user after several incorrect codes.					
	The application does not reveal whether an email address or account exists.					
	Each verification or backup code is random, hard to guess, one-time and expires quickly.					

References

Aljedaani, W., Habib, A., Aljohani, A., Eler, M., Feng, Y., 2024. Does ChatGPT generate accessible code? Investigating accessibility challenges in LLM-generated source code. In: Proceedings of the 21st International Web for All Conference. In: W4A '24, Association for Computing Machinery, New York, NY, USA, ISBN: 9798400710308, pp. 165–176. http://dx.doi.org/10.1145/3677846.3677854, URL https://doi.org/10.1145/3677846.3677854.

Andročec, D., 2026. Vibe coding and web application security: A twin-prompt study. arXiv:2608.20963.

Andruccioli, M., Bassi, B., Delnevo, G., Salomoni, P., 2025. Leveraging large language models for sustainable and inclusive web accessibility. Big Data Cogn. Comput. 9 (10), 247.

Black, G.S., Rimal, B.P., Vaidyan, V.M., 2024. Balancing security and correctness in code generation: An empirical study on commercial large language models. IEEE Trans. Emerg. Top. Comput. Intell. 9 (1), 419–430.

Caldwell, B., Cooper, M., Reid, L.G., Vanderheiden, G., Chisholm, W., Slatin, J., White, J., 2008. Web content accessibility guidelines (WCAG) 2.0. WWW Consort. (W3C) 290 (1–34), 5–12.

Daroff, R.B., Aminoff, M.J. (Eds.), 2014. Encyclopedia of the Neurological Sciences. Academic Press, Oxford, U.K..

Dohmke, T., Iansiti, M., Richards, G., 2023. Sea change in software development: Economic and productivity analysis of the ai-powered developer lifecycle. arXiv preprint arXiv:2306.15033.

Dora, S., Lunkad, D., Aslam, N., Venkatesan, S., Shukla, S.K., 2025. The hidden risks of llm-generated web application code: A security-centric evaluation of code generation capabilities in large language models. In: International Conference on Information Systems Security. Springer, pp. 27–37.

GitHub, 2026. Codeql. URL https://github.com/github/codeql.

González, E.A., Rothkopf, R., Lerner, S., Polikarpova, N., 2025. Hilde: Intentional code generation via human-in-the-loop decoding. In: 2025 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC). pp. 222–233. http://dx.doi.org/10.1109/VL-HCC65237.2025.00032.

Hajipour, H., Hassler, K., Holz, T., Schönherr, L., Fritz, M., 2024. Codelmsec benchmark: Systematically evaluating and finding security vulnerabilities in black-box code language models. In: 2024 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML). IEEE, pp. 684–709.

IBM, 2025. Cost of a Data Breach Report 2025. Standard, IBM.

International Organization for Standardization (ISO) / International Electrotechnical Commission (IEC), 2023. Systems and Software Engineering—Systems and Software Quality Requirements and Evaluation (SQuaRE): Product Quality Model. Standard ISO/IEC 25010:2023, International Organization for Standardization, Geneva, Switzerland.

Japskua, 2026. Paper_llm_for_secure_inclusive_auth. GitHub repository. https://github.com/Japskua/paper_llm_for_secure_inclusive_auth.

Kävrestad, J., Rambusch, J., Nohlberg, M., 2024. Design principles for cognitively accessible cybersecurity training. Comput. Secur. 137, 103630.

Kharma, M., Choi, S., AlKhanafseh, M., Mohaisen, D., 2025. Security and quality in llm-generated code: A multi-language, multi-model analysis. arXiv preprint arXiv:2502.01853.

Koo, T.K., Li, M.Y., 2016. A guideline of selecting and reporting intraclass correlation coefficients for reliability research. J. Chiropr. Med. (ISSN: 1556-3707) 15 (2), 155–163. http://dx.doi.org/10.1016/j.jcm.2016.02.012, URL https://www.sciencedirect.com/science/article/pii/S1556370716000158.

Lennartsson, M., Kävrestad, J., Nohlberg, M., 2021. Exploring the meaning of usable security–a literature review. Inf. Comput. Secur. 29 (4), 647–663.

Liebel, G., Langlois, N., Gama, K., 2024. Challenges, strengths, and strategies of software engineers with ADHD: A case study. In: Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Society. pp. 57–68.

Naqvi, B., Clarke, N., Porras, J., 2021. Incorporating the human facet of security in developing systems and services. Inf. Comput. Secur. 29 (1), 49–72.

Naqvi, B., Kavrestad, J., 2025. Embracing Neurodiversity in Cybersecurity: What Lies Ahead? . Computer (ISSN: 1558-0814) 58 (07), 130–133. http://dx.

doi.org/10.1109/MC.2025.3555463, URL https://doi.ieeecomputersociety.org/10.1109/MC.2025.3555463.

Naqvi, B., Kävrestad, J., Islam, A.N., 2024. Inclusive and accessible cybersecurity: Challenges and future directions. Computer 57 (6), 73–81.

Naqvi, B., Smolander, K., 2024. Practitioners' perspectives on and prospects for usable security. Computer 57 (10), 66–74. http://dx.doi.org/10.1109/MC.2024.3420092.

OWASP Foundation, 2021. OWASP top ten. https://owasp.org/www-project-top-ten/. (Accessed 26 October 2025).

Pasuksmit, J., Takerngsaksiri, W., Thongtanunam, P., Tantithamthavorn, C., Zhang, R., Wang, S., Jiang, F., Li, J., Cook, E., Chen, K., Wu, M., 2025. Human-in-the-loop software development agents: Challenges and future directions. In: 2025 IEEE/ACM 22nd International Conference on Mining Software Repositories. MSR, pp. 756–757. http://dx.doi.org/10.1109/MSR66628.2025.00112.

Ralph, P., Robbes, R., 2020. The ACM SIGSOFT paper and peer review quality initiative: Status report. ACM SIGSOFT Softw. Eng. Notes 45 (2), 17–18.

Ramírez, L.C., Limón, X., Sánchez-García, Á.J., Pérez-Arriaga, J.C., 2024. State of the art of the security of code generated by LLMs: A systematic literature review. In: 2024 12th International Conference in Software Engineering Research and Innovation. CONISOFT, pp. 331–339. http://dx.doi.org/10.1109/CONISOFT63288.2024.00050.

Renaud, K., Coles-Kemp, L., 2022. Accessible and inclusive cyber security: A nuanced and complex challenge. SN Comput. Sci. 3 (5), http://dx.doi.org/10.1007/s42979-022-01239-1.

Semgrep, 2026. Semgrep. URL https://github.com/semgrep/semgrep.

Sergeyuk, A., Zakharov, I., Koshchenko, E., Izadi, M., 2026. Human-AI experience in integrated development environments: a systematic literature review. Empir. Softw. Eng. 31 (3), 55.

Shahid, W.B., Naqvi, B., Afzal, H., 2025. Llmpathy: A multi-agent LLM approach for eliciting inclusive security requirements. In: 2025 IEEE 33rd International Requirements Engineering Conference Workshops. REW, pp. 243–248. http://dx.doi.org/10.1109/REW66121.2025.00037.

Shrout, P.E., Fleiss, J.L., 1979. Intraclass correlations: uses in assessing rater reliability. Psychol. Bull. 86 (2), 420–428. http://dx.doi.org/10.1037/0033-2909.86.2.420.

Suh, H., Tafreshipour, M., Malek, S., Ahmed, I., 2025. Human or llm? a comparative study on accessible code generation capability. arXiv preprint arXiv:2503.15885.

Takerngsaksiri, W., Pasuksmit, J., Thongtanunam, P., Tantithamthavorn, C., Zhang, R., Jiang, F., Li, J., Cook, E., Chen, K., Wu, M., 2025. Human-in-the-loop software development agents. In: 2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP). pp. 342–352. http://dx.doi.org/10.1109/ICSE-SEIP66354.2025.00036.

The MITRE Corporation, 2026. Common weakness enumeration (CWE). URL https://cwe.mitre.org/. (Accessed 02 2026).

Webropol, 2025. Webropol – survey and reporting platform. n.d. https://webropol.co.uk/. (Accessed 29 November 2025).

World Health Organization, 2023. Disability. https://www.who.int/news-room/fact-sheets/detail/disability-and-health. (Accessed 15 September 2025).

Xue, W., Wang, Y., 2025. Human-in-the-loop intelligent testing for safety-critical software. In: 2025 IEEE 36th International Symposium on Software Reliability Engineering Workshops. ISSREW, pp. 171–172. http://dx.doi.org/10.1109/ISSREW67781.2025.00068.

Zhao, J., Sun, Y., Huang, C., Liu, C., Guan, Y., Zeng, Y., Liu, Y., 2025. Towards secure code generation with LLMs: A study on common weakness enumeration. IEEE Trans. Softw. Eng. 51 (12), 3507–3523. http://dx.doi.org/10.1109/TSE.2025.3619281.